

냄새 나침반 — 전체 설계·구현 지침서 겸 기술 보고서

이 문서의 목적 이 저장소를 한 번도 본 적 없는 사람(또는 AI)이 **이 문서 하나만 보고** 같은 웹 서비스를 처음부터 다시 만들 수 있게 하는 것이 목표입니다. 기획 배경, 지켜야 할 원칙, 데이터 형식, 계산 규칙, 모델, 화면 배치·문구, 저장소, 운영자 기능, 테스트·검증 기준, 배포, 알려진 한계까지 모두 적었습니다.

- 기준 시점: 2026-09-19 (작업 브랜치 `main`, 커밋 `d6ee5d1` 이후 미커밋 변경 포함)
- 표기: "반드시"는 바꾸면 안 되는 규칙, "권장"은 같은 결과를 내면 다르게 구현해도 되는 부분입니다.
- 모든 시각은 한국 표준시(KST, `Asia/Seoul`)입니다.

목차

0. 한 장 요약
1. 배경·문제·목표 (기획)
2. 절대 원칙 — 정직성·개인정보 규칙
3. 시스템 전체 구성
4. 기술 스택과 버전
5. 저장소 디렉터리 구조 (파일별 역할)
6. 설정·비밀값·환경변수
7. 데이터 — 원본 워크북, 변환, CSV, 좌표, 경계
8. 도메인 규칙 — 시간, 집계, 상태 어휘, 유형, 방향
9. AI 모델 (`odor_model`) — 학습·평가·추론
10. 공통 추정 서비스 (`core/analysis_service.py`)
11. 예보 (기상청 단기예보 / 시연 예보)
12. 자동 대응 판단 (`core/automation.py`)
13. 주민 조회 앱 "냄새 나침반" — 지도 컴포넌트 전체 명세
14. 문구·용어 사전
15. 주민 기록 앱 "냄새 기록"
16. 관측 저장소 (로컬 SQLite / Supabase)
17. 운영자 앱
18. 보안·개인정보
19. 테스트와 검증 기준
20. 실행·배포
21. 설계 결정 기록 (왜 이렇게 했는가)
22. 알려진 한계와 미완료 항목
23. 1차 기획서(9/15) 대비 변경점

부록

- 부록 A. 짓다 기획서 양식 요약본
- 부록 B. 지도 컴포넌트 HTML 골격 (DOM id 목록)
- 부록 C. 재구축 순서 체크리스트

0. 한 장 요약

항목	내용
이름	냄새 나침반 (Odor Compass) — 운양동 주민 참여형 악취 진단 AI
대상 지역	경기도 김포시 운양동. 지도 범위는 김포시 전체(행정경계)
사용자	① 냄새를 느끼거나 창문을 열기 전 확인하려는 주민 ② 정기 관측에 참여하는 주민 20명 내외 ③ 비공개 운영자(팀)
해결하려는 불편	냄새가 언제, 어느 방향에서 오는지 알 수 없어 대응이 늦고, 기록이 없어 원인 후보를 좁힐 수 없음
핵심 아이디어	주민의 "냄새 있음/없음" 기록 + 기상(풍향·풍속 등) + 후보 지역 좌표를 결합 → 지도 위에 "어느 방향에서, 어떤 유형의 냄새가 들어올 가능성이 있는지"를 표시하고 예보 구간까지 제공
앱 3개 (같은 저장소)	<code>viewer_app.py</code> 주민 조회(지도) · <code>measurement_app.py</code> 주민 기록 · <code>admin_app.py</code> 운영자(비공개, 비밀번호)
프레임워크	Python 3.11 + Streamlit. 지도는 Leaflet 1.9.4를 넣은 양방향 Streamlit 커스텀 컴포넌트 (npm 빌드 없음, 순수 HTML/JS)
AI	scikit-learn 로지스틱 회귀(집단 감지 가능성) + 로지스틱 회귀(냄새 유형 3분류). 상태 "예비 실험·미보정"
데이터	가상 생성 3개월 시나리오 <code>운양동_악취관측_20260618_20260916.xlsx</code> (관측 7,143건, 시간창 364개). 실제 주민 제출은 별도 저장소
가장 중요한 규칙	확률(%)·발생원 단정 금지 / 자료 없음·자료 부족·방향 오류·냄새 없음을 서로 다른 상태로 표시 / 시연·예보·지난 기록을 "실시간"으로 표시하지 않음

1. 배경·문제·목표 (기획)

1.1 문제 선정 배경

- 운양동 주민이 간헐적으로 악취를 느낍니다. 주변에는 **여러 후보**가 있습니다: 고양 구산동 축산지역, 고양 하수처리시설 주변, 김포 하수처리계통(걸포동·레코파크 방면), 김포 하성·월곶 축산지역.
- 간헐적 악취는 **체감 시점과 현장 확인 시점이 어긋나기 쉽습니다**. 주민 생활권의 관측으로 기존 시설 모니터링을 보완할 필요가 있습니다.
- 공개 자료로 확인된 배경: 김포시 2025년 운양동 하수 악취 저감사업 확대·레코파크 복합악취/풍향·풍속 모니터링 발표, 고양 구산동 2026년 스마트 악취 모니터링 도입 보도. 이 자료는 "후보가 여럿이고 관측이 필요하다"는 근거일 뿐이며, **구산동 냄새가 운양동에 도달한다는 증거가 아닙니다**.

1.2 SDGs와 탐구 질문

- SDG 11(지속가능한 도시와 공동체), 세부목표 11.6(도시 환경영향·대기질)과 연결합니다.
- 악취 기록은 **대기질 공식 지표나 건강 위해성 측정값이 아닙니다**.
- 탐구 질문: "운양동 주민의 악취 감지 기록을 기상·공간정보와 결합하면 감지 조건과 유입 가능 방향을 설명하고, 새로운 시점의 감지 위험을 추정할 수 있는가?"

1.3 가설

1. 악취 감지 비율은 풍향·풍속·시간대에 따라 다르다.
2. 후보 지역과 관측 구역 사이의 바람 정렬도와 과거 기상조건은 감지 패턴과 관련된다.
3. 기상정보를 쓰는 모델은 사전에 정한 단순 기준(항상 0, 항상 감지, 직전 창 유지)보다 새로운 날씨의 집단 감지 여부를 더 잘 구분한다. 표본이 부족하면 검증을 유보한다.

1.4 제품 목표 (완성 상태)

주민이 앱을 열면 **세로 스크롤 없이 한 화면에서** 다음 네 가지를 바로 알 수 있어야 합니다. 1. 내가 선택한 위치 (또는 권한으로 확인한 내 위치) 2. 냄새가 어느 방향에서 들어올 가능성이 있는지 (나침반: 들어오는 방향 / 지도: 이동 방향) 3. 어떤 유형의 냄새로 추정되는지 (축산계·하수계·기타·미확인) 4. 어느 시각의 정보인지 (지난 기록 / 최신 / 업데이트 지연 / 예보)

2. 절대 원칙 — 정직성·개인정보 규칙

이 원칙은 기능보다 우선합니다. 재구축 시 **하나라도 어기면 잘못 만든 것입니다.**

2.1 표현 원칙

1. **확률·퍼센트 금지.** 모델 점수 `p_detect` 는 보정되지 않은 값입니다. 주민 화면에는 "유입 가능성 낮음/보통/높음"만 보여주고, `p값·%정렬도` 점수를 브라우저에 **보내지도 않습니다**(페이로드 단계에서 제거).
2. **발생원 판정 금지.** "측산계 추정"은 냄새 느낌의 분류이지 특정 시설이 원인이라는 뜻이 아닙니다. 후보 지역 이름은 **흐름선 상세 패널에서만** 보이고, 첫 화면에는 방향만 표시합니다.
3. **모든 모델 결과에 "예비 실험·미보정" 배지**를 작게, 항상 붙입니다. 모델이 없으면 그 자리에 "추정 보류"를 표시합니다.
4. **상태를 섞지 않습니다.** 다음은 모두 다른 상태이며, 어떤 것도 "냄새 없음"이나 "안전"으로 바꿔 표시하지 않습니다. - 자료 없음 — 그 시각·구역에 기록 자체가 없음 / 기상 결측 - 자료 부족 — 기록은 있으나 판단 가능한 응답이 3명 미만 - 방향 보류 — 약풍(0.5 m/s 미만)이라 방향을 정할 수 없음 - 추정 보류 — 모델 파일 없음·로딩 실패·핵심 기상 결측 - 감지 낮음 — 유효 응답 3명 이상이고 감지율 20% 미만 (이것만이 "적게 감지됨"을 뜻함)
5. **시간 상태를 구분합니다.** - 지난 기록 — 사용자가 직접 과거 시각을 고름 (오류나 지연이 아님) - 최신 — 최신 관측 시각이고 1시간 이내 - 업데이트 지연 — 최신을 보는데 최신 자료가 1시간 넘게 오래됨 - 예보 / 시연 예보 — 최신 이후 예보 구간 - 어떤 경우에도 "실시간"이라는 단어를 쓰지 않습니다.
6. **시연 자료 표시 유지.** 데모 모드(`DATA_MODE=demo`)에서는 위치 카드에 시연 배지를 항상 표시합니다. 기상 자료는 원본의 `weather_basis` 에 따라 관측 / 추정 / 시연 으로 구분해 보여줍니다.
7. **없는 데이터를 만들지 않습니다.** 시간 슬라이더는 실제 존재하는 관측창 시각만 씁니다. 결측을 보간하지 않습니다. 자료가 없는 시각에는 이전 화살표를 남겨두지 않고 지웁니다.
8. **바람만으로 유색 약취 화살표를 만들지 않습니다.** 근거가 약하면 중립 회색 "바람 이동" 선만 그립니다 (규칙은 §10.6).
9. **단일 관측소 바람으로 소용돌이·확산 경로를 꾸며내지 않습니다.** 흐름선은 한 풍향을 여러 지점에 반복해 그린 "방향 참고선"입니다.
10. **가짜 기능 금지.** 실제 연결이 없는 기기에 대해 전원 토글, "연결 성공" 표시를 만들지 않습니다. "연결된 기기 없음", "기기 연결 · 준비 중"(비활성 버튼)만 표시합니다.
11. **실패를 숨기지 않습니다.** 기상청 예보 호출이 실패하면 시연 예보로 대체하되 실패 사유를 "자료 정보"와 알림에 남깁니다. 저장 실패 시 저장된 것처럼 보이지 않습니다.

2.2 개인정보 원칙

- 실명·주소·연락처·GPS 좌표를 **수집·저장하지 않습니다**. 참여자는 무작위 코드(예: R07)와 관측 구역만 입력합니다.
- 브라우저(주민 조회 화면)에는 **집계값만** 보냅니다. 참여자 코드, 개별 기록, 후보 점수는 보내지 않습니다.
- "내 위치"는 사용자가 버튼을 누른 뒤에만 브라우저 권한을 요청합니다. 선택 좌표는 화면 계산에만 쓰고 관측 DB에 저장하지 않습니다.
- 참여자 코드는 비밀번호가 아니므로, 기록 앱의 "내 최근 기록"은 **현재 세션에서 제출한 기록만** 보여줍니다.
- 운영자 앱은 비밀번호(PBKDF2 해시)로 잠그며, 공개 앱에서 링크하지 않습니다.

3. 시스템 전체 구성

3.1 세 개의 앱

진입 파일	이름	공개	역할
viewer_app.py (app.py는 이것을 import해 실행)	냄새 나침반	공개	전체 화면 지도, 시간 슬라이더, 작은 나침반, 위치 선택, 유형 필터, 공기질 관리 패널, 예보 구간
measurement_app.py	냄새 기록	공개	기본값 없는 단계별 관측 입력, 제출 확인, 세션 내 최근 기록
admin_app.py	운영자	비공개 (로컬)	인증 후 기존 분석(H1/H2), AI 실험, 우리 집 대응(기기 시연), ⑦ 예측 모델 (평가·재학습·모델 선택·운영 설정·임시 좌표), 설정·CSV 관리

3.2 데이터 흐름

```
[원본 워크북.xlsx] --core.import_workbook--> data/provided/{reports,weather,windows,provenance}.csv
|
[주민 제출] --measurement_app--> 관측 저장소(SQLite 로컬 or Supabase) |
v
core.public_data.dashboard(mode) / window_slots(mode)
|
weather -----+ | reports, zones, windows(집계)
v v
core.analysis_service.weather_inputs(weather, 시간창 목록)
|
analysis_service.predict_windows() <-- odor_model(joblib) + data/geo/model_geo.js
on + data/model_settings.json
|
forecast_inputs() (기상청 or sample_forecast.csv) -> predict_windows()
|
core.map_data.prepare_payload() (집계 + 예측 + 화살표 규칙 -> 프레임 배열)
|
core.map_component.odor_map (Leaflet 컴포넌트, iframe)
|
사용자 조작(위치·시각·필터·패널) -> state 값을 Python으로 반환 -> rerun
```

핵심 설계: 지도 화살표, 나침반, 요약 카드, 공기질 관리, 운영자 화면은 모두 `predict_windows()` 한 함수의 결과만 씁니다. 화면마다 따로 계산하지 않습니다. 화살표를 그릴지 말지(arrow 종류)도 Python에서 한 번 계산해 프레임에 넣고, 브라우저는 그 값으로 그리기만 합니다.

3.3 데이터 모드

- `DATA_MODE=demo` (기본): 읽기 전용 시나리오(`data/provided/*.csv`)만 표시합니다. 주민 제출과 섞지 않습니다.
- `DATA_MODE=live`: 관측 저장소의 주민 제출을 표시합니다. 이때 실제 기상 연동이 없으면 기상은 비어 있고, 방향과 추정은 보류됩니다. **시연 기상을 실제 기상 대신 쓰지 않습니다.**

4. 기술 스택과 버전

requirements.txt (그대로 사용):

```
streamlit>=1.39,<2
pandas>=2.1
numpy>=1.26
scipy>=1.11
scikit-learn==1.8.0 # odor_model/artifacts/odor_model.joblib was pickled with 1.8.0
joblib>=1.3
plotly>=5.20
pydeck>=0.9
pyserial>=3.5
pytest>=8.0
openpyxl>=3.1
requests>=2.32
```

- **scikit-learn**은 반드시 **1.8.0**으로 고정합니다. 번들 모델이 1.8.0으로 pickle되어 있어서 1.5.x에서는 불러오기는 되지만 예측할 때 `AttributeError: 'LogisticRegression' object has no attribute 'multi_class'` 가 납니다. 모델을 새로 학습하면 그 환경의 버전에 맞춰집니다.
- pandas 3.x도 지원해야 합니다. 날짜 해상도가 `datetime64[s] / [us]` 로 섞이면 `merge_asof` 가 실패하므로, 병합 키는 `astype("datetime64[ns]")` 로 맞추니다(§8.8).
- 브라우저 검증용: `playwright + python -m playwright install chromium` (requirements에는 없음).
- 지도: Leaflet 1.9.4를 `components/odor_map/vendor/` 에 **파일로 포함**합니다(CDN 불필요). 배경 타일은 OpenStreetMap 표준 타일(`https://tile.openstreetmap.org/{z}/{x}/{y}.png`)이며 인터넷이 필요합니다.
- 글꼴: Pretendard(jsDelivr CSS), 실패 시 시스템 sans-serif.
- Python 3.11 권장(Streamlit Community Cloud에서 3.11 선택). 3.10에서도 동작 확인.

5. 저장소 디렉터리 구조 (파일별 역할)

```
app.py                # from viewer_app import main; main() - 기존 배포 진입점 호환
viewer_app.py         # 주민 조회 앱 (지도 컴포넌트 호스트)
measurement_app.py    # 주민 기록 앱
admin_app.py          # 운영자 앱 (인증 필수)
requirements.txt
README.md / DEPLOYMENT.md
.streamlit/config.toml # 테마 색·headless
.streamlit/secrets.example.toml # 비밀값 예시(실제 secrets.toml은 Git 제외)
assets/logo.svg      # 나침반 모양 로고
components/odor_map/
  index.html          # 컴포넌트 DOM (부록 B)
  map.css             # 레이아웃·반응형
  map.js              # 상태·렌더링·Streamlit 프로토콜
  vendor/leaflet.js, leaflet.css, LICENSE # Leaflet 1.9.4
core/
  config.py           # KST, 정기 관측 시각, 구역 공개 이름, 설정 읽기
  public_data.py      # dashboard(mode), window_slots(mode)
  map_data.py         # 조회 앱 페이로드 생성, 위치 검증(resolve_selection)
  map_component.py    # components.declare_component('odor_map', path=components/odor_map)
  analysis_service.py # ★ 공통 추정 서비스 (predict_windows 등)
  automation.py       # 자동 대응 판단(ON/OFF/HOLD)
  observations.py     # 주민 관측 표준 레코드, Local/Shared 저장소
  import_workbook.py  # 원본 xlsx → data/provided CSV 변환
  features.py         # (기존 분석용) 관측창 집계 make_windows, window_id_time
  geo.py              # haversine_km, initial_bearing, source_geometry
  analysis.py         # (운영자) 8방위 감지율, 카이제곱, 정렬도 요약
  model.py            # (운영자 "AI 실험") 기존 모델 학습·저장
  risk.py             # (운영자) 방위 이름, 규칙 기반 점수, 방향 힌트
  repo.py, schema.py  # (운영자) SQLite 저장소와 스키마
  auth.py             # PBKDF2 비밀번호 해시·검증·로그인 화면
  ui.py, public_ui.py # 운영자·기록 앱 공용 CSS·카드·배지·나침반 SVG
  sample_data.py      # 원본이 없을 때 쓰는 예시 데이터 생성기
odor_model/
  __init__.py         # ★ 약취 예측 모델 패키지
  features.py         # 구역×시간창 학습표, 특징, 후보 방위·정렬도, DEFAULT_GEO
  train.py            # 학습·시간순 평가 → artifacts/
  predict.py          # predict_rows, 기상청 예보 조회, 예보 CSV 읽기, CLI
  sample_forecast.csv # 예보 입력 예시(가상값) = "시연 예보" 원천
  README.md
  artifacts/odor_model.joblib, evaluation.md, candidates.csv, zone_windows.csv, sample_forecast_resul
t.csv
data/
  provided/운양동_약취관측_20260618_20260916.xlsx # 원본 워크북(재학습 입력)
  provided/reports.csv, weather.csv, windows.csv, provenance.csv # 변환 결과
  sample/zones.csv, sources.csv, weather_kimpo_aws.csv, reports.csv, source_geometry_by_zone.csv
  geo/gimpo.geojson  # 김포시 행정경계(KOSTAT 2018)
  geo/provenance.json # 경계 출처
  geo/model_geo.json  # ★ 모델용 구역 대표점·후보 좌표(임시)
  model_settings.json # ★ 운영 설정(임계값·약풍·자동 대응 규칙·활성 모델)
  models/            # (Git 제외) 운영자 재학습 모델 data/models/odor/<시각>/
  resident.db, odor.db # (Git 제외) 로컬 SQLite
deployment/supabase.sql # 공유 저장소 테이블·RLS
ext/                 # 기기 시연: device_sim.py, arduino.py, arduino/odor_fan.ino
```

```
scripts/fetch_map_assets.py # Leaflet-경계 재다운로드(앱 실행 시에는 사용 안 함)
tests/                      # pytest + Playwright 브라우저 검증
```

.gitignore에는 artifacts/ (스크린샷 폴더)가 있으므로 **반드시** !odor_model/artifacts/ 를 추가해 모델 산출물이 커밋에서 빠지지 않게 합니다.

6. 설정·비밀값·환경변수

설정 읽기 규칙(`core/config.py`): - `setting(name, default)` — 환경변수가 `st.secrets` 보다 우선합니다. - `nested_setting(section, key, default)` — `st.secrets[section][key]`, 환경변수 `SECTION_KEY` (대문자)가 우선합니다.

키	기본값	의미
<code>APP_ENV</code>	<code>cloud</code>	<code>local</code> 이면 로컬 SQLite 저장을 허용. 클라우드에 공유 저장소가 없으면 저장 버튼 비활성
<code>DATA_MODE</code>	<code>demo</code>	<code>demo</code> 시나리오만 / <code>live</code> 주민 제출
<code>MEASUREMENT_APP_URL</code>	<code>""</code>	조화 앱 "냄새 기록" 메뉴가 여는 기록 앱 주소. 비어 있으면 "기록 연결 · 준비 중"
<code>OBSERVATION_DB_PATH</code>	<code>data/resident.db</code>	로컬 관측 DB 경로
<code>ADMIN_DB_PATH</code>	<code>data/odor.db</code>	운영자 분석 DB
<code>[database] url, key / DATABASE_URL, DATABASE_KEY</code>	없음	Supabase REST URL, 서버 전용 service role 키
<code>[admin] password_hash / ADMIN_PASSWORD_HASH</code>	없음	<code>python -m core.auth</code> 로 생성. 없으면 운영자 앱 잠김
<code>[kma] service_key / KMA_SERVICE_KEY</code>	없음	기상청 단기예보(공공데이터포털) 서비스키
<code>[kma] nx, ny / KMA_NX, KMA_NY</code>	55, 128	단기예보 격자 좌표(운양동 근처, 운영자 확인 필요)

`.streamlit/secrets.example.toml`:

```
APP_ENV = "local"
DATA_MODE = "demo"
MEASUREMENT_APP_URL = "http://localhost:8502"

[database]
url = ""
key = ""

[admin]
password_hash = ""

[kma]
service_key = ""
nx = 55
ny = 128
```

`.streamlit/config.toml`: `primaryColor #1F5C8B`, `backgroundColor #F7F9FB`, `secondaryBackgroundColor #EDF2F5`, `textColor #1B2A38`, `headless=true`, `gatherUsageStats=false`.

7. 데이터

7.1 원본 워크북 윤양동_악취관측_20260618_20260916.xlsx

시트 5개: 요약, 관측데이터, 시간창집계, 기상스냅샷, 코드북.

전체 규모: 기간 2026-06-18 ~ 2026-09-16(91일), 참여자 20명(구역 고정), 3구역 × 하루 4회 정기 관측 (07:30-12:30-18:30-22:00). - 전체 기록 7,143 = 정기 7,039 + 자발 제보 104 - 유효 정기 관측 6,822, 정기 관측 감지 936(감지율 13.7%) - 집단 감지 시간창 22 / 전체 시간창 364 - **모든 기록 record_origin = simulated** (가상 주민 기록). scenario_seed: 20260916 = 9/9~9/16 기존 자료, 20260919 = 6/18~9/8 확장 자료.

관측데이터 열 (7,143행):

열	의미 / 값
observation_id	O00001... 기록 ID
participant_id	P001... 익명 참여자(20명)
zone_id	U01, U02, U03 (윤양동 관측 구역)
observed_at, received_at	관측 시각, 서버 접수 시각 (KST)
context	outdoor (전부)
odor_detected	True / False / 빈 값(판단 어려움). 분포: False 5,886, True 1,040, 빈 값 217
intensity	감지 시 1~5, 미감지 0, 판단 어려움 빈 값
odor_type	없음 5,886 · 축산계 추정 369 · 구분 어려움 313 · 하수계 추정 284 · 판단 어려움 217 · 기타 74
collection_mode	scheduled 7,039 / spontaneous 104
prediction_seen	예측 화면을 봤는지
disruption_type	(전부 비어 있음)
window_id	W-YYYYMMDD-HHMM 같은 정기 관측 시간창
weather_station_id	427
temperature_c, humidity_pct, wind_speed_10min_ms, wind_from_sector_10min, wind_from_sector_center_deg, rain_rolling_1h_mm	해당 창외 기상(10분 평균 풍속, 16방위 풍향과 중심각, 직전 1시간 이동 누적 강수)
weather_basis	observed_public_reference / estimated_from_adjacent_actuals / synthetic_climatology
record_origin, scenario_seed	simulated, 시드

시간창집계 열 (364행, 6/18 07:30 ~ 9/16 22:00): window_id, window_start, scheduled_records, valid_records, detected_records, detection_rate, median_intensity, group_label, wind_from_sector_10min, wind_speed_10min_ms, humidity_pct, weather_basis, record_origin. - group_label: 유효 응답 3명 이상이고 감지율 0.5 이상이면 1, 3명 미만이면 빈 값. - weather_basis 분포: synthetic_climatology 332, observed_public_reference 30, estimated_from_adjacent_actuals 2.

기상스냅샷 열 (1,092행 = 364 × 관측소 3곳): station_id, station_name, observed_at, temperature_c, humidity_pct, wind_speed_10min_ms, wind_from_sector_10min, wind_from_sector_center_deg, rain_rolling_1h_mm, lat, lon, weather_basis, source_name, source_url. - 관측소: 427 양촌(주민 기록 결합 기준, 37.66877/126.64721), 441 김포, 589 고양고봉. - source_name: "기상청 AWS 기후 특성 기반 생성값"(생성 구간) 또는 "기상청 날씨누리 지역별상세관측"(9/9~9/16 공개 관측값).

검증 기준으로 쓰는 세 시간창 (반드시 기억):

시각	풍향 / 풍속	관측	기대 화면
2026-09-11 22:00	동남동(112.5°) 0.2 m/s, basis=관측	창 전체 10/20 감지(0.5, 집단 감지). Z2 5/7	마커 "집단 감지", 화살표 없음 "방향 보류", 나침반 바늘 비활성
2026-09-15 22:00	동북동(67.5°) 0.5 m/s, 관측	창 11/19(0.5789). Z2 3/7	화살표 서남서 247.5°, 축산계 주황, 나침반 동북동
2026-09-14 12:30	서북서(292.5°) 3.0 m/s, 관측	감지 0건	유색 화살표 없음, 회색 바람 레이어만

7.2 변환: python -m core.import_workbook <xlsx> [--output data/provided]

record_origin 이 simulated 외의 값이면 예외를 냅니다.

구역 매핑: U01→Z1, U02→Z2, U03→Z3.

냄새 유형 매핑 (원본 → 앱 내부): 없음→없음, 판단 어려움→모름, 구분 어려움→모름, 축산계 추정→축산분뇨, 하수계 추정→하수, 기타→기타, 그 외 모름.

reports.csv (7,143행) 열: report_id (=observation_id), observer_code (P001→R01 형식, 숫자만 뽑아 2자리), zone_id (Z*), observed_at, submitted_at (=received_at, 둘 다 YYYY-MM-DD HH:MM:SS), report_mode (scheduled→scheduled, spontaneous→extra), odor (True→1, False→0, 빈 값→빈 값), intensity (빈 값→0), odor_type (위 매핑), environment (outdoor/indoor), confidence (odor 빈 값이면 low, 아니면 mid), saw_forecast (0/1), memo ="제공 시나리오 · 실제 관측 아님", record_origin =simulated, idempotency_key = scenario:<id>, window_id (원본 그대로 W-...).

weather.csv (364행, 관측소 427만): station_id, weather_at, wd(=wind_from_sector_center_deg), ws, temp, humidity, pressure(빈 값), rain, quality_flag="ok", weather_basis.

windows.csv (364행): window_id, window_start, valid_records, detected_records, detection_rate, group_label, weather_basis, record_origin. → 시간 슬라이더의 시각 목록입니다.

provenance.csv: source_file, record_origin=simulated, scenario_seed="20260916 · 20260919", report_rows=7143, weather_rows=364, station_id=427, notice="제공 파일 자체가 simulated로 표시되어 실제 주민 관측으로 사용하지 않음".

7.3 구역·후보 좌표

data/sample/zones.csv (지도 표시용, 앱 ID):

zone_id	name	rep_lat	rep_lon	공개 이름(ZONE_PUBLIC_NAMES)
Z1	운양동 북부(한강변·라베니체 일대)	37.662	126.679	한강변·라베니체 인근
Z2	운양역 중심부	37.654	126.680	운양역 인근

zone_id	name	rep_lat	rep_lon	공개 이름(ZONE_PUBLIC_NAMES)
Z3	운양동 남부(모담산·학교 일대)	37.647	126.685	모담산·학교 인근

boundary_note 는 "대표점 — 추후 실측 교체". 화면에서는 항상 공개 이름을 씁니다.

data/sample/sources.csv (운영자 기존 분석용): D1 고양 일산서구 구산동·법곳동 일대(축산, 37.679/126.698), D2 김포 하성면·월곶면 일대(축산, 37.72/126.63), D3 김포 레코파크·계양천 방면(하수·분뇨처리, 37.633/126.700).

data/geo/model_geo.json (모델용, 저장소 좌표를 우선, provisional: true):

```
{
  "provisional": true,
  "note": "임시 좌표 · 구역 대표점은 data/sample/zones.csv, 후보 좌표는 data/sample/sources.csv에서 가져옴 (C4는 모델 기본값). 운영자 검증 필요.",
  "zones": {
    "U01": {"name": "한강변·라베니체 인근", "app_zone_id": "Z1", "lat": 37.662, "lon": 126.679},
    "U02": {"name": "운양역 인근", "app_zone_id": "Z2", "lat": 37.654, "lon": 126.68},
    "U03": {"name": "모담산·학교 인근", "app_zone_id": "Z3", "lat": 37.647, "lon": 126.685}
  },
  "candidates": {
    "C1": {"name": "고양 구산동 축산지역", "type": "축산계", "lat": 37.679, "lon": 126.698, "app_source_id": "D1"},
    "C2": {"name": "김포 하성·월곶 축산지역", "type": "축산계", "lat": 37.72, "lon": 126.63, "app_source_id": "D2"},
    "C3": {"name": "김포 하수처리계통(걸포동)", "type": "하수계", "lat": 37.633, "lon": 126.7, "app_source_id": "D3"},
    "C4": {"name": "고양 하수처리시설 주변", "type": "하수계", "lat": 37.648, "lon": 126.748}
  }
}
```

- 모델 ID(U01~U03, C1~C4)는 학습 특징(zone_U01, align_C1 등)과 묶여 있어 **바꾸면 안 됩니다**. 이름·좌표만 편집할 수 있습니다.
- 후보 이름은 "지역·시설 유형" 수준으로만 씁니다(특정 사업장 상호 금지).
- 참고: 패키지 기본값 DEFAULT_GEO 는 구역 이름 "운양동 북측/중앙/동측", 좌표 U01 37.6720/126.6790, U02 37.6660/126.6825, U03 37.6690/126.6900, C1 37.6960/126.7120, C2 37.7250/126.6050, C3 37.6380/126.7020, C4 37.6480/126.7480입니다. 번들 모델은 이 기본 좌표로 학습했으므로, 저장소 좌표로 추론하면 정렬도 특징이 조금 달라집니다. 좌표를 확정된 뒤 재학습하는 것을 권장합니다.
- 운영자 앱에서 구역 좌표를 저장하면 data/sample/zones.csv 의 rep_lat/rep_lon 도 같이 갱신합니다 (지도와 모델 좌표 일치).

7.4 행정경계 data/geo/gimpo.geojson

- 출처: southkorea/southkorea-maps 의 KOSTAT 2018 skorea-municipalities-2018-geo.json 에서 properties.name == "김포시" 하나만 뽑은 FeatureCollection (MultiPolygon, 코드 31230).
- provenance.json: source URL, publisher "southkorea/southkorea-maps · KOSTAT", year 2018, license "KOSTAT: Free to share or remix (upstream README)", notice "2018 공개 행정경계 · 최신 법정 경계 아님".
- scripts/fetch_map_assets.py 로 다시 받을 수 있습니다(Leaflet 1.9.4 js/css/LICENSE도 같이). 앱 실행 중에는 내려받지 않습니다.

7.5 운영 설정 `data/model_settings.json`

```
{
  "active_model": "bundled",
  "thr_mid": 0.5,
  "thr_high": null,
  "calm_ms": 0.5,
  "align_min": 0.5,
  "align_tie": 0.15,
  "obs_rate_min": 0.25,
  "auto": {"start_level": "높음", "start_count": 2, "stop_level": "낮음", "stop_count": 2}
}
```

키	의미
active_model	<code>bundled</code> (<code>odor_model/artifacts</code>) 또는 <code>data/models/odor/<run></code> 의 run 이름. 운영자가 명시적으로 적용할 때만 바뀜
thr_mid	이 값 이상이면 "보통"
thr_high	이 값 이상이면 "높음". <code>null</code> 이면 활성 모델의 검증 임계값(<code>thr_valid</code> , 번들 0.75)
calm_ms	이 풍속 미만이면 방향 보류
align_min	정렬 후보로 인정하는 최소 정렬도
align_tie	상위 두 후보 정렬도 차가 이보다 작으면 "여러 방향 가능"
obs_rate_min	추정 낮음이면서 실제 감지율이 이보다 낮으면 회색 바람 레이어만
auto	자동 대응 히스테리시스 규칙

모두 "운영 설정 · 공인 기준 아님"입니다. 파일이 없거나 깨지면 위 기본값을 씁니다. `auto` 안의 값은 기본값 위에 병합합니다.

8. 도메인 규칙

8.1 시간

- 저장·표시 모두 KST. `now_kst()` 는 `datetime.now(ZoneInfo("Asia/Seoul"))`.
- 정기 관측 시각 `SCHEDULED_TIMES` = `["07:30", "12:30", "18:30", "22:00"]`, 허용 오차 ± 15 분.
- `classify_collection_mode(moment)`: 허용 범위 안이면 ("scheduled", "현재 정기 관측 시간입니다."), 아니면 ("spontaneous", "추가 제보로 기록됩니다.").
- `observation_window(moment)`: 가장 가까운 정기 시각이 ± 15 분 안이면 그 시각, 아니면 30분 단위로 내림한 시각.
- 1차 기획서의 21:30 관측은 22:00으로 바뀌었습니다(워크북과 일치).

8.2 관측창 시각 해석 `window_id_time(values)` (`core/features.py`)

- 워크북 ID `^W-(\d{8}-\d{4})$` → `%Y%m%d-%H%M` 으로 파싱.
- 그 외(라이브 기록의 ISO 문자열·Timestamp): 시간대 정보가 있으면 KST로 변환 후 시간대 제거, 없으면 그대로(이미 KST). 실패하면 NaT.
- 결과는 `datetime64[ns]`.

8.3 구역×관측창 집계 규칙 (조회 앱)

- 기록의 관측창 시각 = `window_id_time(window_id)`, 없으면 `observed_at` 을 30분 단위로 반올림.
- 집계 대상: `report_mode == "scheduled"` 그리고 `environment == "outdoor"` 그리고 `observer_code != "GUEST"`.
- 같은 참여자·같은 구역·같은 창 의 중복은 `submitted_at` 이 가장 늦은 것(마지막 정정값)만 남김.
- 유효 응답 = `odor` 가 0 또는 1인 기록(판단 어려움 제외). `n` = 유효 응답의 고유 참여자 수, `d` = `odor==1` 개수, `rate` = `d/n` (`n=0`이면 없음). `rate`는 소수 셋째 자리로 반올림합니다.
- 상태: - 그 구역·창에 기록이 **하나도 없으면** 자료 없음 - `n < 3` → 자료 부족 - `rate ≥ 0.5` → 집단 감지 - `rate ≥ 0.2` → 일부 감지 - 그 외 → 감지 낮음
- 관측 유형(`type`): 감지 기록의 `odor_type`을 분류(§8.4)해 셉니다. 최다 유형이 **2건 이상이고 단독 최다** (동률 아님)일 때만 그 유형, 아니면 `unknown`.
- `odor_evidence` = (`n ≥ 3` and `d ≥ 2`).
- `median` = 유효 응답 강도 중앙값(없으면 없음).
- 추가 신고·실내·판단 어려움은 감지율의 분모·분자에 넣지 않습니다.

8.4 냄새 유형 분류

- 관측 기록(`classify_type`): 텍스트에 분뇨 또는 축산 이 있으면 `livestock`, 하수가 있으면 `sewage`, 기타·탄내·화학물질과 비슷함 이면 `other`, 그 외 `unknown`.
- 모델 유형(`type_top`): 축산계 추정→`livestock`, 하수계 추정→`sewage`, 기타/미확인→`unknown`.
- 유형 키 → 라벨·색 (`TYPES`, 반드시 이 색):

키	라벨	색	선 패턴(과거)
livestock	축산계	#D97706 주황	실선
sewage	하수계	#7C3AED 보라	긴 점선 12 7

키	라벨	색	선 패턴(과거)
other	기타	#0F766E 청록	짧은 점선 3 6
unknown	미확인	#64748B 회색	점선 7 7

- 색은 **유형**을 뜻합니다. 위험 수준은 별도 배지(유입 가능성 낮음/보통/높음)로 표시합니다. 색만으로 구분하지 않도록 라벨과 선 패턴을 함께 씁니다.

8.5 관측 상태 색 (구역 마커)

hold(자료 없음/자료 부족) #9AA5AE, 집단 감지 #C0563B, 일부 감지 #D99A3C, 감지 낮음 #3D8B5F. 예보 구간 프레임에서는 모든 마커가 #9AA5AE (관측 없음).

8.6 방향 규칙 (가장 자주 틀리는 부분)

- 기상청 풍향 `wind_from_deg` 는 바람이 **불어오는** 방향입니다(북=0°, 시계방향).
- 냄새가 **이동하는** 방향 `move_to_deg` = $(wind_from_deg + 180) \% 360$.
- 지도 화살표** = 이동 방향(`move_to_deg`), 라벨 "이동 방향".
- 나침반 강조 바늘** = 들어오는 방향(`wind_from_deg`), 라벨 "들어오는 방향".
- 예: 북동풍 45° → 지도 화살표 남서 225°, 나침반 북동 45°.
- 16방위 이름: 북, 북북동, 북동, 동북동, 동, 동남동, 남동, 남남동, 남, 남남서, 남서, 서남서, 서, 서북서, 북서, 북북서 (index = $\text{round}(\text{deg}/22.5) \% 16$; 모델 쪽은 $\text{int}(((\text{deg}+11.25)\%360)/22.5)$ 로 같은 결과).
- 나침반 바늘 회전은 **최단 호**로 보간합니다: $\text{deltaAngle}(a,b) = ((b-a+540)\%360)-180$, 누적 각도 `needleAngle += deltaAngle(current%360, target)`. 359°→1°는 +2°만 회전합니다.
- GPS는 위치 확인에만 쓰고, 휴대폰 방향(나침반 센서)은 추적하지 않습니다. 나침반은 **북쪽 고정**입니다.

8.7 후보 방위와 정렬도 (odor_model.features)

- `bearing_deg(lat1,lon1,lat2,lon2)`: 구역 대표점에서 후보를 바라보는 초기 방위(대권), 0~360.
- `distance_m`: 하버사인(반지름 6,371,000 m).
- `alignment(wind_from, cand_bearing) = \max(0, 1 - \text{angdiff}/90)`, $\text{angdiff} = |((a-b+180)\%360)-180|$. 45° 차이면 0.5, 90° 이상이면 0.
- 해석: 후보가 구역에서 방위 b에 있을 때 풍향(불어오는 방향)이 b와 가까울수록 "그 후보 쪽에서 바람이 불어온다" = 정렬.
- 정렬도는 풍향에서 파생된 값일 뿐, 발생원의 독립 증거가 아닙니다.

8.8 pandas 날짜 해상도

- 병합 전 양쪽 키를 `astype("datetime64[ns]")` 로 맞춥니다(`core/features.make_windows` 의 `merge_asof`). pandas 3에서 `[s]` 와 `[us]` 가 섞여 실패하는 문제를 막습니다.

9. AI 모델 (odor_model/)

9.1 모델이 답하는 것

1. **감지 여부**: 구역별로 "이 기상이면 주민 절반 이상이 냄새를 감지할 가능성" $p \rightarrow$ 낮음 / 보통($\geq \text{thr_mid}$ 0.5) / 높음($\geq \text{thr_high}$ 0.75).
2. **어디서 오는지**: 풍향과 각 구역에서 본 후보 방위의 정렬도 $\rightarrow$ 정렬 / 여러 방향 가능 / 정렬 후보 없음 / 방향 보류(약풍).
3. **어떤 냄새로 느껴질지**: 축산계 추정 / 하수계 추정 / 기타/미확인 (주민 주관 분류를 재현하는 보조 모델. 약함).
4. **미래**: 기상청 단기예보(VEC 풍향·WSD 풍속·REH 습도·TMP 기온·PCP 강수)를 같은 입력으로 넣으면 시각별 예측. `basis=kma_forecast`.

9.2 학습표 `build_zone_windows(obs, min_valid=3)`

- 입력: 관측데이터 시트.
- 정기(`collection_mode=="scheduled"`) + 실외(`context=="outdoor"`)만 사용합니다.
- `det = odor_detected`가 "true"면 1, 그 외 값은 0, 빈 값은 제외.
- (window_id, zone_id)별: `n_valid`, `n_detected`, `detection_rate`, `label = 1 if rate $\geq$ 0.5 else 0` (`n_valid` $\geq$ 3일 때만, 아니면 NaN), `top_type` (감지 기록 최다 유형), 그리고 창 의 첫 행 기상(`temperature_c`, `humidity_pct`, `wind_speed`, `wind_from_deg`, `rain_1h_mm`, `weather_basis`).
- `window_start = window_id`의 YYYYMMDD-HHMM.
- `add_lag_features`: 같은 구역의 직전 창 `prev_rate`, `prev_label` (비교 기준선 "지속성"과 선택 모델에만 사용).
- 결과: 1,092행(364창 $\times$ 3구역), 양성 77(7.1%).

9.3 특징 `make_features(df, geo, use_lag=False)` — 입력 열 순서 그대로 20개

```
wind_sin, wind_cos      # 풍향(불어오는) 라디안의 sin/cos
wind_speed              # m/s
calm                    # wind_speed < 0.5  $\rightarrow$  1
wind_u = -speed*sin(wd) # 이동 방향 벡터(동쪽 +)
wind_v = -speed*cos(wd) # 이동 방향 벡터(북쪽 +)
humidity, temperature
rain                    # rain_1h_mm >= 1.0  $\rightarrow$  1
hour_sin, hour_cos      # (시 + 분/60) / 24 주기
doy_sin, doy_cos        # 연중 일자 / 365 주기
zone_U01, zone_U02, zone_U03 # 원-핫
align_C1, align_C2, align_C3, align_C4 # 풍향-후보 방위 정렬도($8.7), 값 없으면 0
```

- 같은 창의 관측값(감지 여부·강도·유형·감지자 수)은 **입력에서 제외**합니다(정답 누설 방지).
- `use_lag=True` 이면 `prev_rate` (NaN $\rightarrow$ 0)를 추가합니다(비교용 모델에만).

9.4 시간순 분할 (사전 고정, 무작위 분할 금지)

구간	기간	건수(양성)
train	2026-06-18 ~ 2026-08-25	828 (54)

구간	기간	건수(양성)
valid	2026-08-26 ~ 2026-09-08	168 (15)
test	2026-09-09 ~ 2026-09-16 (공개 관측 기상 구간)	96 (8)

같은 시각의 모든 구역은 같은 구간에 들어갑니다.

9.5 모델

- **주 모델:** `Pipeline(StandardScaler → LogisticRegression(C=0.5, class_weight="balanced", max_iter=2000))`, 기상 특징만.
- **비교 1:** `RandomForestClassifier(n_estimators=400, min_samples_leaf=5, class_weight="balanced_subsample", random_state=7)`.
- **비교 2:** 로지스틱 회귀 + `prev_rate` (nowcast 전용).
- **임계값:** 사전 고정 0.5, 그리고 **검증 자료에서만** 0.20~0.80을 0.05 간격으로 훑어 F1 최대인 값 `thr_valid` (결과 0.75). 평가 자료는 보지 않습니다.
- **유형 분류기:** 정기 관측 중 감지된 개인 기록(`odor_detected true`)으로 학습합니다. 목표 = `type_target(odor_type)` (축산계 추정/하수계 추정 외는 기타/미확인). 같은 특징, `StandardScaler → LogisticRegression(C=0.5, max_iter=2000)`. 학습은 train 기간, 평가는 test 기간.
- **저장** (`artifacts/odor_model.joblib`, dict): `detect_model`, `detect_model_rf`, `detect_model_lag`, `type_model`, `type_classes` (['기타/미확인', '축산계 추정', '하수계 추정']), `feature_columns` (위 20개), `geo`, `thr_fixed=0.5`, `thr_valid=0.75`, `splits`, `trained_on`, `status="예비 실험·미보정"`, `train_positive_rate=0.0705`.
- **함께 저장:** `evaluation.md`, `zone_windows.csv`, `candidates.csv` (구역→후보 방위·거리표).

9.6 번들 모델 평가 결과 (test 9/9~9/16, 96건, 양성 8)

모델	임계값	정확도	정밀도	재현율	F1	AUC	혼동행렬
기준선: 항상 0	0.50	0.917	계산 불가	0.000	0.000	0.500	TP0 FP0 FN8 TN88
기준선: 항상 감지	0.50	0.083	0.083	1.000	0.154	0.500	TP8 FP88 FN0 TN0
기준선: 지속성	0.50	0.854	0.125	0.125	0.125	0.523	TP1 FP7 FN7 TN81
로지스틱(기상)	0.50	0.417	0.113	0.875	0.200	0.756	TP7 FP55 FN1 TN33
로지스틱(기상)	0.75	0.635	0.135	0.625	0.222	0.756	TP5 FP32 FN3 TN56
랜덤 포레스트	0.50	0.896	0.250	0.125	0.167	0.680	TP1 FP3 FN7 TN85
로지스틱(기상+직전 창)	0.50	0.427	0.115	0.875	0.203	0.753	TP7 FP54 FN1 TN34

- 해석: 지속성 기준선보다 사건을 덜 놓치지만(재현율 0.63 vs 0.13) 오경보가 많습니다. 유형 분류기 평가 정확도 0.378(다수 클래스 0.366)으로 약합니다.
- 계수 상위: `rain -1.51`, `humidity +0.79`, `wind_speed -0.69`, `wind_sin -0.68` ...
- 6/18~9/8 구간은 기상과 주민 기록이 모두 생성 자료이고, 9/9~9/16은 공개 관측 기상 + 시뮬레이션 주민 기록입니다. 따라서 수치는 **파이프라인 검증용**이며 실제 성능이 아닙니다.
- 분모가 0이면 "계산 불가"로 적습니다.

9.7 추론 `predict_rows(rows, m, settings=None)`

입력 행: `window_start`, `wind_from_deg`, `wind_speed`, `humidity_pct`, `temperature_c`, `rain_1h_mm` (+ 선택 `basis`, `source`). 각 행을 3개 구역으로 펼칩니다.

`settings`: `thr_mid` (기본 0.5), `thr_high` (기본 `m["thr_valid"]`), `calm_ms` (0.5), `align_min` (0.5), `align_tie` (0.15).
없는 키는 기본값을 씁니다.

행별 처리: 1. `wind_from_deg`, `wind_speed`, `humidity_pct`, `temperature_c` 중 하나라도 결측이면 → `status="추정 오류"`, `reason="핵심 기상 결측: <열들>".` 2. 아니면 (모든 행×구역을 **한 번에** 특징 생성.예측, 결과는 행별 처리와 동일): - `p = detect_model.predict_proba(X[:,1], level = 높음 if p≥thr_high else 보통 if p≥thr_mid else 낮음.` - `type_probs` (3클래스, 소수 3자리), `type_top = argmax.` - `wind_from_sector`, `move_to_deg=(wd+180)%360`, `move_to_sector`, `wind_speed.` - 방향 판단: - `ws < calm_ms` → `direction_status="방향 오류(악풍)"`, 후보 없음. - 아니면 그 구역의 후보별 `align_score` 계산, 내림차순, `align_score ≥ align_min` 만 남김: - 없으면 정렬 후보 없음 - 2개 이상이고 1:2위 차 < `align_tie` 면 여러 방향 가능 - 그 외 정렬 - `aligned_candidates = "이름 (유형,0.78); 이름(유형,0.59)"` 형식. - `status="예비 추정".`

출력 열: `window_start`, `zone_id`, `zone_name`, `basis`, `source`, `status`, `(reason)`, `p_detect`, `level`, `wind_from_deg`, `wind_from_sector`, `move_to_deg`, `move_to_sector`, `wind_speed`, `type_top`, `type_probs`, `direction_status`, `aligned_candidates`, `direction_note`.

- 성능: 364행 기준 행별 예측 32.6초 → 일괄 예측 4.1초. 번들 `sample_forecast_result.csv` 와 결과가 완전히 같음을 확인했습니다.
- import 경로: 패키지 안에서는 `from .features import ...`, 스크립트로 직접 실행할 때를 위해 `except ImportError: from features import ...`.

9.8 CLI

```
python -m odor_model.predict now --time "2026-09-19 22:00" --wind-dir 동북동 --wind-speed 0.8 --humidity 82 --temp 21
python -m odor_model.predict forecast --csv odor_model/sample_forecast.csv [--out out.csv]
python -m odor_model.predict kma --key <서비스키> --nx 55 --ny 128
python -m odor_model.train --data data/provided/운양동_악취관측_20260618_20260916.xlsx --out <폴더> --geo data/geo/model_geo.json
```

`--wind-dir` 은 16방위 이름 또는 도(°)를 받습니다.

9.9 예보 입력 형식

- `read_forecast_csv(path)`: 열 이름 별칭을 허용합니다(`datetime|time|window_start|fcst_time`, `wind_from_deg|vec|wind_dir`, `wind_speed|wsd|wind_speed_ms`, `humidity_pct|reh|humidity`, `temperature_c|tmp|temp`, `rain_1h_mm|pcp|rain`). 풍향 방위 이름은 도로 바꾸고, `basis` 기본값은 `forecast_csv` 입니다.
- `sample_forecast.csv` (가상값, "시연 예보"):

```
datetime,wind_from_deg,wind_speed,humidity_pct,temperature_c,rain_1h_mm
2026-09-19 18:00,270,2.4,58,24.1,0
2026-09-19 21:00,67.5,0.9,79,20.8,0
2026-09-20 00:00,45,0.3,88,19.2,0
2026-09-20 03:00,22.5,0.7,91,18.4,0
2026-09-20 06:00,45,1.1,90,18.0,0
```

```
2026-09-20 09:00,90,2.2,66,22.5,0
2026-09-20 12:00,112.5,3.4,47,26.3,0
2026-09-20 15:00,135,3.0,49,26.8,0
2026-09-20 18:00,157.5,1.6,64,23.9,1.5
2026-09-20 21:00,180,0.8,86,20.2,3.0
```

- `fetch_kma_forecast(service_key, nx=55, ny=128, base=None)`:
 - 호출: `http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getVilageFcst, dataType=JSON, numOfRows=1000`.
 - 발표 시각: 02:05:08·11:14:17·20:23시(10분 이후 제공). 현재 시각 이전 가장 최근 발표를 고르고, 없으면 전날 23시.
 - 결과를 `fcstDate+fcstTime` 별로 VEC/WSD/REH/TMP/PCP로 펼칩니다. PCP "강수없음"은 0이고, "mm"."미만" 문자는 제거합니다.
 - `basis="kma_forecast", source="단기예보 YYYY-MM-DD HH00 발표"`.
 - `base` 는 **KST naive datetime**으로 넘겨야 합니다(클라우드 서버는 UTC).
-

10. 공통 추정 서비스 `core/analysis_service.py`

규칙: 지도·나침반·공기질 관리·운영자 화면은 이 모듈의 결과만 씁니다. 앱 코드에서 `odor_model.predict` 를 직접 부르지 않습니다.

10.1 상수

```
BUNDLED_MODEL      = odor_model/artifacts/odor_model.joblib
BUNDLED_EVALUATION = odor_model/artifacts/evaluation.md
RETRAINED_DIR      = data/models/odor
SETTINGS_PATH      = data/model_settings.json
GEO_PATH           = data/geo/model_geo.json
SAMPLE_FORECAST    = odor_model/sample_forecast.csv
MODEL_STATUS       = "예비 실험·미보정"
BASIS_LABELS = {observed_public_reference:"관측", estimated_from_adjacent_actuals:"추정",
                 synthetic_climatology:"시연", kma_forecast:"예보", forecast_csv:"시연 예보"}
TYPE_KEYS = {"축산계 추정":"livestock", "하수계 추정":"sewage", "기타/미확인":"unknown"}
LEVELS = ("낮음", "보통", "높음")
```

10.2 함수 명세

함수	동작
<code>load_settings()</code>	기본값을 깊은 복사한 뒤 파일 값을 덮어씁니다. <code>auto</code> 는 키 단위로 병합합니다. 파일 오류는 무시합니다
<code>save_settings(values)</code>	JSON(UTF-8, indent 2, ensure_ascii False)으로 저장합니다
<code>load_geo()</code> / <code>save_geo(geo)</code>	<code>model_geo.json</code> 을 읽고 씁니다. 없으면 <code>DEFAULT_GEO</code> . 저장할 때 <code>zones.csv</code> 대표점도 동기화합니다
<code>app_zone_ids()</code>	<code>{U01:"Z1", ...}</code>
<code>retrained_runs()</code>	<code>data/models/odor/*/*odor_model.joblib</code> 가 있는 폴더 이름(최신순)
<code>model_path(name)</code>	<code>bundled</code> → 번들 경로, 그 외 → <code>RETRAINED_DIR/name/odor_model.joblib</code>
<code>load_model(name=None) → (model or None, info)</code>	파일이 없으면 <code>reason="모델 없음"</code> . 있으면 <code>_load(path, mtime, geo_json)</code> (<code>lru_cache 4개</code>). 모든 예외는 <code>reason="모델 로딩 실패 · <예외 클래스명>"</code> . 성공 시 <code>info={ok:True, trained_on, thr_valid, name, path, status}</code>
<code>_load</code>	<code>joblib</code> 로드 → 모델의 <code>align_*</code> 후보 ID와 <code>U01~U03</code> 이 <code>geo</code> 에 모두 있으면 <code>model["geo"]</code> 를 저장소 <code>geo</code> 로 교체 → 시험 예측 1건 (2026-09-15 22:00, 67.5°, 1.0 m/s, 80%, 20°C, 0 mm). <code>sklearn</code> 버전이 맞지 않으면 여기서 실패하고, 주민 요청 중에는 실패하지 않습니다
<code>weather_inputs(weather, times)</code>	앱 기상표(<code>weather_at</code> , <code>wd</code> , <code>ws</code> , <code>temp</code> , <code>humidity</code> , <code>rain</code> , <code>weather_basis</code> , <code>quality_flag</code>) → 모델 입력. 시각마다 "그 시각 이전 1시간 안의 가장 최근, <code>quality_flag ok</code> " 1건을 씁니다. 없으면 <code>basis="missing"</code> , 값 없음
<code>predict_windows(weather_df, model_name=None)</code>	빈 입력이면 빈 표. 모델이 없으면 <code>_held()</code> (모든 행×구역 <code>status="추정 보류"</code> , <code>reason=<info.reason></code> , 풍향-풍속은 보존). 있으면 <code>predict_rows(weather_df, model, None이 아닌 설정값)</code> . 예외는 <code>reason="모델 오류 · <클래스>"</code> . 공통 추가 열: <code>app_zone_id</code> , <code>model_status</code> , <code>basis_label</code> , <code>type_key</code> , <code>candidates</code> (<code>{(name,type)}</code> — 점수 제거)

함수	동작
<code>forecast_inputs(now=None) → (rows, meta)</code>	\$11
<code>arrow_kind(pred, wind, observed_rate, settings)</code>	\$10.6
<code>compact(row)</code>	예측 행 → 브라우저용 dict(\$10.5)
<code>day_phrase(moment, now)</code>	"오늘/내일/모레/MM.DD" + "새벽(0~5)/오전(6~11)/오후(12~17)/밤(18~23)"

10.3 후보 문자열 → 목록

정규식 `([^\;]+?)\((([^\,()]+),([^\d.]+)\)` 로 이름(유형,점수) 를 뽑고 점수는 버립니다. 결과 `[{"name":"고양 구산동 축산지역","type":"축산계"}, ...]`.

10.4 캐시 (viewer_app.py)

- `model_key()` = JSON[활성 모델 경로, 파일 mtime(없으면 0), 설정 전체, geo 전체]. 모델 파일을 지우거나 설정.좌표를 바꾸면 키가 바뀌어 다시 계산합니다.
- `observed_predictions(inputs, key): st.cache_data(max_entries=8)`. 입력 DataFrame 내용으로 해시합니다.
- `forecast(key): st.cache_data(ttl=3600)` — 기상청 호출을 최대 1시간 캐시합니다.
- `map_payload(mode, key): st.cache_data(ttl=15)` — 주민 제출을 15초마다 반영합니다.

10.5 브라우저용 예측 dict (compact)

- 추정 보류: `{"status":"추정 보류","reason":"모델 없음"}`
- 예비 추정:

```
{
  "status": "예비 추정",
  "level": "높음",
  "from_sector": "동북동",
  "direction": "여러 방향 가능",
  "type": "livestock",
  "type_label": "축산계 추정",
  "candidates": [
    {
      "name": "고양 하수처리시설 주변",
      "type": "하수계"
    },
    {
      "name": "고양 구산동 축산지역",
      "type": "축산계"
    }
  ]
}
```

- `candidates` 는 비어 있으면 넣지 않습니다. `p_detect`, `type_probs`, 정렬 점수는 절대 넣지 않습니다. 자료 구분(basis)은 프레임 단위에 한 번, 모델 상태는 페이로드에 한 번만 넣습니다(용량 절약).

10.6 화살표 종류 규칙 arrow_kind (지도·나침반·요약이 공유)

입력: 이 구역의 `pred`, 프레임 `wind` ({from,to,speed,at} 또는 없음), `observed_rate`. `observed_rate` 는 구역 유효 응답이 3명 이상이면 구역 감지율, 아니면 시간창 전체 감지율(`windows.csv detection_rate`), 둘 다 없으면 없음입니다.

순서대로 판정합니다. 1. `wind` 없음 → `none` (자료 없음. 화살표 없음, 나침반 "방향 미확인") 2. `wind.speed < calm_ms` 또는 `pred.direction == "방향 보류(약풍)"` → `calm` (화살표 없음, 마커에 "방향 보류", 나침반 바늘 끄) 3. `pred` 없음 또는 `pred.status != "예비 추정"` (모델 없음 등) → `wind` (회색 바람 이동) 4. `pred.level == "낮음"` 이고 (`observed_rate` 없음 또는 `< obs_rate_min(0.25)`) → `wind` 5. 그 외 → `odor` (유형 색 유입 추정)

→ 결과: 9/11 22:00은 calm, 9/15 22:00 Z2는 odor(높음·축산계), 9/14 12:30은 wind(낮음·감지 0).

10.7 페이로드 `prepare_payload(reports, weather, zones, windows, demo, slots, predictions, forecast, forecast_predictions, model_info, settings)`

프레임 시각: `slots (windows.csv)`가 있으면 그 364개 시각만 씁니다. 없으면(live) 기록 창 시각과 집계 창 시각의 합집합입니다.

관측 프레임:

```
{"at": "2026-09-15T22:00:00+09:00", "kind": "observed", "basis": "관측",
 "wind": {"from": 67.5, "to": 247.5, "speed": 0.5, "at": "2026-09-15T22:00:00+09:00"},
 "window_rate": 0.5789,
 "zones": {"Z2": {"n": 7, "detected": 3, "status": "일부 감지", "type": "unknown", "rate": 0.429,
 "odor_evidence": true, "median": 0.0, "pred": {...}, "arrow": "odor"}, "Z1": {...}, "Z3": {...}}}
```

- `wind`: 그 시각 이전 1시간 안의 가장 최근 ok 기상. 풍향·풍속이 결측이면 없음(null).
- `basis`: 기상 `weather_basis`의 라벨. 바람이 없으면 "자료 없음". `basis` 값이 없으면 demo면 시연, live면 관측.
- 시각 문자열은 KST ISO(+09:00).

예보 프레임: 마지막 관측 시각보다 늦은 예보 행만 씁니다. `kind: "forecast"`, `basis: meta.label (예보 / 시연 예보)`, `window_rate: null`, 모든 구역 관측 통계는 자료 없음(`n=0`), `pred / arrow`는 예보 입력 예측으로 채웁니다.

페이로드 최상위:

```
frames[], latest_index (마지막 관측 프레임 인덱스), demo(bool), now(ISO),
zones: [{id, name(공개 이름), lat, lon}], boundary(geojson), types(TYPES),
forecast: {source: "kma" | "csv" | "none", label, issued, error}, calm_ms,
model: {ok, reason, status: "예비 실험·미보정"},
outlook: {"Z1": "09.16 22:00 높음 · 오늘 밤 높음(예보)", ...},
info: {자료 유형, 기간, 기록 수, 주민 기록, 기상 자료, 기상 관측소, 예보, 추정 모델, 행정경계, 관측 위치}
```

- demo일 때 `info` 예시: 자료 유형 시연 · 기간 2026.06.18-2026.09.16 · 기록 수 7,143건 · 주민 기록 가상 생성 · simulated · 기상 자료 시연 332건 · 관측 30건 · 추정 2건 · 기상 관측소 427 양촌 · 예보 시연 예보 · 예시 파일(실패 시 · 기상청 예보 연결 실패 · `ConnectionError` 추가) · 추정 모델 예비 실험·미보정 또는 추정 보류 · 모델 없음 · 행정경계 KOSTAT 공개자료 · 2018 · 관측 위치 구역 대표점 · 임시.
- 전체 크기: 약 0.5~0.8 MB (374프레임). 필요 없는 필드는 넣지 않습니다.
- **개인정보 확인:** 페이로드 JSON에 `observer_code`, R01 같은 코드, `p_detect`, `type_probs`, `align_score`, %가 없어야 합니다.

10.8 공기질 관리 요약 `outlook(frames, zone_ids, now)`

- **현재 부분:** 마지막 관측 프레임의 구역 예측 level. 그 시각이 `now`와 1시간 이내면 "지금 낮음", 아니면 "09.16 22:00 높음".
- **예보 부분:** 기준 시각 `anchor = now` 이후의 첫 예보 시각(없으면 첫 예보 시각). `anchor`부터 6시간 안의 예보 프레임 중 **최고 level**, 그 level이 처음 나오는 시각의 `day_phrase` → "오늘 밤 높음(예보)".
- 둘을 " · "로 잇고, 둘 다 없으면 "추정 보류".

10.9 위치 선택 검증 `resolve_selection(payload, state)`

- 기본 위치 [37.654, 126.680] (운양역 인근). 숫자가 아니거나 김포 경계 밖이면 기본값으로 되돌립니다.

- 김포 경계 판정: GeoJSON MultiPolygon에 대한 ray casting(외곽 링 안, 구멍 밖).
 - 가장 가까운 구역 대표점이 **2 km 이내**면 그 구역, 아니면 `zone_id=None`, 이름 "관측 구역 밖".
 - 시각: state의 `at` 과 같은 프레임, 없으면 `latest_index` (마지막 관측 프레임. 예보 프레임이 아님).
 - 반환: `{position, zone_id, zone_name, distance_km, at, index, sequence}`.
-

11. 예보 (기상청 단기예보 / 시연 예보)

`forecast_inputs(now=None)`: 1. `kma.service_key` 가 있으면 `fetch_kma_forecast(key, nx, ny, base=now(KST, tz 제거))` 를 호출합니다. `now` 를 시(hour) 단위로 내린 시각 이후 행만 씁니다. - 결과가 있으면 `meta={source:"kma", label:"예보", issued:"단기예보 2026-09-19 1700 발표", error:""}`. - 결과가 비면 `error="기상청 예보 없음"`, 예외가 나면 `error="기상청 예보 연결 실패 · <클래스>"`. 2. 키가 없거나 실패하면 `sample_forecast.csv` 를 읽고 `source="시연 예보 파일", meta={source:"csv", label:"시연 예보", issued:"", error:<있으면 유지>}`. 3. 예시 파일도 읽을 수 없으면 (빈 표, `{source:"none", label:"예보 없음", error:"예보 파일 오류 · ..."}`).

화면 규칙: - 예보 프레임은 **둥근 점선**(`dashArray "1 9", lineCap round`)과 점선 화살표 머리로 그려 과거 관측(실선·유형 패턴)과 섞이지 않게 합니다. - 시간 바 배지는 시연 예보 또는 예보, 자료 구분 배지는 예보 기준, 나침반 하단은 예보 기준입니다. - 사용자가 예보 구간으로 이동했는데 `forecast.error` 가 있으면 알림에 "<오류> · 시연 예보" 를 띄웁니다. - 슬라이더 트랙에서 `latest_index` 이후 구간은 옅은 남보라(`#e7e9f7`) 배경으로 칠합니다. - 헤더 보조 문구 예보 ~09.20 21:00. 예보가 없으면 미래 자료 없음(모바일에서는 숨김). - 예보 구간에서도 "실시간"이라고 쓰지 않습니다.

12. 자동 대응 판단 `core/automation.py`

판단만 합니다. 기기를 제어하지 않습니다.

```
def decide(levels, rule, enabled=True) -> (action, reason):
    if not enabled: return 'HOLD', '사용 안 함'
    if 마지막 start_count개가 모두 start_level('높음'): return 'ON', '높음 2회 연속'
    if 마지막 stop_count개가 모두 stop_level('낮음'): return 'OFF', '낮음 2회 연속'
    return 'HOLD', '조건 미충족'
```

- `levels`: 구역의 관측 시각 예측을 시간순으로 나열합니다. 추정 보류는 `None` 이므로 연속이 끊깁니다.
- 히스테리시스: ON과 OFF 사이에서는 HOLD(이전 상태 유지)입니다.
- `zone_levels(predictions, zone_id)`: 앱 ID(Z1)와 모델 ID(U01)를 모두 받습니다.
- `auto_actions(predictions, enabled) → {Z1:{action,reason}, ...}`. 조회 앱은 매 실행마다 `state.auto_enabled` (기본 True)로 계산해 컴포넌트 인자 `auto` 로 넘깁니다.
- **기기 모듈용 인터페이스**: `get_auto_action(zone_id, predictions=None, enabled=True) -> "ON" | "OFF" | "HOLD"`. `predictions`가 없으면 현재 데이터 모드로 직접 계산합니다.
- 규칙 상수는 `data/model_settings.json` 의 `auto` 에 있고, 운영자 앱에서 연속 횟수를 바꿀 수 있습니다.

13. 주민 조회 앱 "냄새 나침반" — 지도 컴포넌트 전체 명세

13.1 Streamlit 호스트 (viewer_app.py)

1. `st.set_page_config(page_title='냄새 나침반', page_icon='assets/logo.svg', layout='wide', initial_sidebar_state='collapsed')`.
2. 전역 CSS로 Streamlit 헤더·툴바·메뉴·푸터를 숨기고, 본문 패딩 0, 블록 간격 0, `[data-stale="true"] {opacity:1}` (재실행 중 흐려짐 방지), 컴포넌트 `iframe height:100dvh; width:100%; border:0; 페이지 overflow:hidden`.
3. `payload, predictions = map_payload(data_mode(), model_key())`. 예외가 나면 `st.error('서버 연결 실패') + 다시 시도 버튼만 보여줍니다`.
4. `state = st.session_state.map_state(없으면 {})`, `selection = resolve_selection(payload, state)`, `auto = auto_actions(predictions, enabled=state.get('auto_enabled', True))`.
5. `odor_map(payload=..., selection=..., initial_state=state, auto=auto, measurement_url=..., key='resident_map', default=None)`.
6. 반환값의 `sequence` 가 저장된 값보다 크면 `st.session_state.map_state = result` 후 `st.rerun()`.

`core/map_component.py`: `components.declare_component('odor_map', path=<repo>/components/odor_map)` — 빌드 없는 정적 폴더 컴포넌트입니다.

13.2 컴포넌트 통신 프로토콜 (map.js)

- 로드 즉시 `parent.postMessage({isStreamlitMessage:true, type:'streamlit:componentReady', apiVersion:1}, '*')`.
- `message` 이벤트에서 `event.source === window.parent` 그리고 `type === 'streamlit:render'` 일 때 `args` 를 받습니다(`payload, selection, initial_state, auto, measurement_url`).
- Leaflet 전역 `L` 이 없으면 타일 오류 카드에 "지도 초기화 실패"를 띄우고, 다시 시도 버튼으로 새로고침합니다.
- 첫 호출이면 `initialize(args)`. 이후 호출은 `args.selection.sequence >= state.sequence` 일 때만 `payload`·위치·시각을 반영합니다(오래된 서버 응답이 새 선택을 덮어쓰지 않게). 표시 시각을 최신으로 강제로 옮기지 않습니다.
- 매번 `streamlit.setFrameHeight = window.innerHeight`.
- 값 보내기 `send()`: 현재 지도 중심·zoom을 `state`에 넣고 `sequence += 1` 후 `streamlit.setComponentValue {value: state, dataType:'json'}`.
- 창 크기가 바뀌면 `map.invalidateSize()` 후 높이를 다시 보냅니다.

13.3 컴포넌트 상태 state

```
position: [lat, lon]           # 기준 위치
position_kind: 'selected'|'gps' # gps일 때만 "내 위치"라고 부름
at: ISO 문자열                 # 선택 시각
mode: 'latest'|'past'|'forecast'
filter: 'all'|'livestock'|'sewage'|'other'|'unknown'
panel: ''|'air'|'record'|'help'|'zone'|'flow'|'info'|'criteria'
auto_enabled: bool (기본 true)
compass_collapsed: bool
```

```
center: [lat, lon], zoom: number
sequence: int (증가만)
```

초기값: {position: selection.position, position_kind:'selected', at: selection.at, mode:'latest', filter:'all', panel:'', auto_enabled:true, sequence:0, ...initial_state}. 지도 이동·확대만으로는 기준 위치가 바뀌지 않습니다. 재생하거나 시간을 바꿔도 중심·줌·위치는 그대로입니다.

13.4 지도 초기화

- `L.map('map', {zoomControl:false, maxBounds: 김포경계.getBounds().pad(0.10), maxBoundsViscosity:1, minZoom:10, maxZoom:17, zoomSnap:0.5})`.
- 경계 밖은 좁은 여유(10%)만 둡니다. 전국·수도권으로 멀어지지 않고, 구산동 같은 가까운 외부 후보는 보입니다.
- 원거리 시설 때문에 지도를 자동으로 넓히지 않습니다.
- 초기 보기: `state.center || [37.654,126.680]`, `state.zoom || 13.5`.
- 타일: OSM `https://tile.openstreetmap.org/{z}/{x}/{y}.png`, `maxZoom 19`, attribution @ OpenStreetMap (링크), `referrerPolicy:'strict-origin-when-cross-origin'`. 타일 레이어에 CSS `filter: saturate(.48) brightness(1.03)` (채도 낮은 차분한 배경).
- 김포 경계: `color #477ba4`, `weight 2`, `fillColor #d3e4ed`, `fillOpacity .07`, `dashArray '6 5'`, 클릭 불가.
- 레이어 순서: 흐름선(flows) → 구역 마커(markers) → 기준 위치 핀(zIndexOffset 1000).
- 기준 위치 핀: `divIcon className 'selected-marker' 18×18`, 파란 원 `#1f5c8b` + 흰 테두리 3px + 옅은 파란 후광. 상시 툴팁은 오른쪽(offset [12,0], 굵은 파란 글씨). 문구는 기준 위치(gps면 내 위치). 클릭하면 구역 상세 패널을 엽니다.
- 타일 오류: `tileerror` 가 2번 이상이면 오류 카드 지도 연결 실패 / 인터넷 연결을 확인하세요 / [다시 시도]. 다시 시도는 실패 횟수를 초기화하고 `tiles.redraw()`. 성공 로드 시 카드를 숨깁니다. 빈 지도로 방치하지 않습니다.

13.5 화면 배치 — 데스크톱 (폭 701px 이상)

전체 `#shell` = 높이 100dvh, 가로 flex.

요소	위치·크기	내용
왼쪽 세로 메뉴 <code>#nav</code>	폭 76px, 흰 배경, 오른쪽 테두리. 버튼 높이 72px, 글자 12px, 아이콘 26px	맨 위 로고 SVG(38px, 55px 영역) · 지도 · 공기질 관리 · + 냄새 기록 · ? 도움말 · 맨 아래 냄새 나침반 (12px 회색)
지도 <code>#map</code>	남은 공간 전체(<code>position:absolute; inset:0</code>)	Leaflet
위치 카드 <code>.location-card</code>	좌상단 left 20, top 20, 폭 290, 패딩 14/16	1줄: 김포 · 냄새 나침반 (12px 회색) + 오른쪽 시연 배지 (demo만) / 2줄: 파란 점 + 구역 공개 이름(16px 굵게) / 3줄: 내 위치 + 위치 선택 버튼(13px) / 4줄(구분선 아래): 요약 <code>#summary</code>
유형 필터 카드 <code>.filters</code>	위치 카드 오른쪽(간격 14), 패딩 8/12	작은 라벨 냄새 유형 · 추정 + 버튼 전체 / 축산계 / 하수계 / 기타 / 미확인 (각 앞에 6px 유형 색 점, 전체 제외). 선택된 버튼은 파란 글씨·옅은 파란 배경, <code>aria-pressed</code>
지도 조작 <code>#map-controls</code>	오른쪽 right 18, top 20. 버튼 48×48 세로	+ (확대) - (축소) 선택 위치 김포 전체
알림 <code>#notice</code>	left 20, top 224, 최대 폭 340, 왼쪽 파란 3px 선	짧은 알림(role=status)

요소	위치·크기	내용
타일 오류 <code>#tile-error</code>	left 20, top 285, 주황 테두리	위 \$13.4
범례 <code>#legend</code>	left 20, bottom 138, 높이 한 줄	[추정 방향 ①] (누르면 흐름선 상세) · 마커 관측 · 화살표 추정 · 레이어 라벨 <code>#layer-label</code>
나침반 카드 <code>#compass</code>	right 18, bottom 144, 폭 210	\$13.9
상세 패널 <code>#panel</code>	left 20, top 220, 폭 330, 최대 높이 <code>calc(100% - 365px)</code> , 스크롤	지도 요소나 메뉴를 누를 때만 열림. 제목 + × 닫기
시간 바 <code>#timeline</code>	left 20, right 18, bottom 18, 패딩 9/15	\$13.10
Leaflet 저작권	시간 바 위(<code>.leaflet-bottom{bottom:126px}</code>), 10px	Leaflet © OpenStreetMap 유지

공통 스타일: - 글자: Pretendard → 시스템 sans-serif, 기본 14px, 숫자 `tabular-nums`, 본문색 `#1B2A38`, 배경 `#F7F9FB`, 파란 강조 `#1F5C8B`, 선 `#dce4eb`. - 카드: 흰색 97% 불투명, 1px 선, 반경 12px, 그림자 0 2px 9px `#1b2a3810` (작게). - 버튼: 최소 높이 **44px**(터치 영역), 반경 8px, hover `#edf3f8`, 포커스 outline 3px `#1f5c8b`, 비활성 투명도 .45. - 보조 글씨 12~13px, 본문 14~16px. 큰 그라데이션·큰 그림자·거대한 숫자 카드·이모지 메뉴는 쓰지 않습니다. - 배지 `.badge`: 반경 5px, 12px, 패딩 3/6, 기본 `#eef2f5 / #526575`. - `.hold` (자료 없음·부족·보류): 사선 줄무늬 `repeating-linear-gradient(135deg, #edf0f3 0 3px, #dce2e8 3px 5px)` - `.high` `#f7e7e0 / #a14530`, `.medium` `#fff2dc / #93621c`, `.low` `#e8f3ed / #397751` - `.forecast` `#eef0fb / #3d4aa0` + 1px 점선 테두리 `#8b93cf` - `.model` 11px `#f1f5f9 / #5b6b79` - `#multi` (여러 방향 가능) `#fff4e5 / #8a5a12` - `prefers-reduced-motion: reduce` 이면 모든 `transition-animation`을 끕니다.

13.6 화면 배치 — 모바일 (폭 700px 이하, 기준 390×844)

- 왼쪽 메뉴 → 하단 내비게이션: 높이 64px, 가로 4등분, 로고와 하단 문구 숨김, 아이콘 22px, 글자 11px. 본문 `padding-bottom:64px`.
- 위치 카드: 좌우 10px, 폭 100%. 김포 · 냄새 나침반 라벨 숨김, 시연 배지는 오른쪽으로 float. 제목 14px, 버튼과 요약을 한 줄 인라인(12px)으로.
- 유형 필터: 위치 카드 아래(간격 8), 폭 100%, 버튼 균등 분배, 12px.
- 지도 조작: top 206, right 10, 44×44.
- 알람: top 200, left 10, right 64. 타일 오류: top 270, 최대 폭 245.
- 나침반: bottom 133, right 10, 폭 **192**, 가로 배치(나침반 72×80 + 오른쪽 글). 들어오는 방향 라벨과 선택 시각 기준 문구는 숨깁니다. 제목은 말줄임(최대 112px). 접이식(제목 버튼으로 펼침/접기).
- 범례: left 10, bottom 133, 최대 폭 155, 세로 쌓기(10~11px).
- 시간 바: 좌·우·아래 10px. 헤더 12px, 미래 자료 없음/예보 ~... 문구 숨김, 배지 10px.
- 상세 패널 → 바텀시트: top auto, bottom 128, 좌우 10, 최대 높이 58%.
- Leaflet 저작권: bottom 119px, 8px.
- 슬라이더에 `touch-action: pan-x` 를 줘 지도 드래그와 충돌하지 않게 합니다.
- 페이지 가로 스크롤이 없어야 합니다(`scrollWidth ≤ innerWidth`).

13.7 구역 마커

- 각 구역 대표점에 `circleMarker` 반경 9, 흰 테두리 3, 채움색 = 관측 상태색(\$8.5, 예보 구간이면 회색), 불투명 .95.

- 툴팁(위쪽, offset -8): 구역 공개 이름 · 관측 상태 + (arrow=calm이면 · 방향 보류). 예보 구간이면 구역 이름 · 예보. 줌 12.5 이상에서 상시 표시, 방향 보류면 점선 테두리 툴팁. 줌이 바뀔 때마다 다시 그립니다.
- 클릭·Enter·Space: 그 대표점을 기준 위치로 선택하고 구역 상세 패널을 엽니다. `tabindex=0`, `role=button`, `aria-label="<이름> 상세"`.

13.8 흐름선(화살표) 그리기

- 프레임에 `wind` 가 없으면 아무것도 그리지 않습니다(이전 화살표를 지움).
- **선 목록 만들기** (`flowLines`): 1. 기준 위치가 어떤 구역(2 km 이내)에도 속하지 않으면: 바람이 있고 약풍이 아닐 때만 기준 위치에 회색 `wind` 선 최대 3개(수직 오프셋 0, -0.45, +0.45 km). 2. 속하면: 선택 구역을 맨 앞에 두고, 각 구역의 `arrow` 가 `odor` 또는 `wind` 이며 필터에 걸리지 않는 구역마다 대표점을 지나는 선 1개. 3. 선이 3개 미만이면 선택 구역 대표점 기준 ± 0.45 km 평행선을 3개가 될 때까지 추가합니다. 4. 새 선의 기준점이 기존 선과 **바람 방향에 수직인 거리 0.3 km 미만**이면 추가하지 않습니다(겹침 방지). 5. 최대 **4개**.
- 수직 거리는 평면 근사로 구합니다: `dx=(Δlon)·cos(lat)·111.32`, `dy=(Δlat)·110.57`, `across = dx·cos(to) - dy·sin(to)`.
- **각 선**: 기준점에서 `from` 방향으로 0.8 km 지점부터 `to` 방향으로 0.8 km 지점까지(총 1.6 km). 좌표 이동은 대권 공식(반지름 6371.0088 km).
- 흰 외곽선: 굵기 +4, 불투명 .9, 클릭 불가
- 본선: 색·굵기·불투명·점선 규칙(아래), 클릭하면 흐름선 상세
- 화살표 머리 4개: 선을 따라 -0.6, -0.2, +0.2, +0.6 km. `divIcon` (SVG 셰브론 M4 15L10 5L16 15). 흰 7px 외곽선 위에 유형색 3.5px 선, `transform: rotate(<to>deg)`. 크기 12 + 굵기 px(15~17px). 예보 구간이면 머리도 `stroke-dasharray 3 2`. 키보드로 선택할 수 있고 `title`은 유입 추정 상세 / 바람 이동 상세.
- **스타일**:

종류	색	굵기	불투명	dashArray
odor, level 낮음/보통/높음	유형색	3 / 4 / 5	.7 / .85 / .95	livestock 실선, sewage 12 7, other 3 6, unknown 7 7
wind (바람 이동)	#64748B	3	.75	4 8
예보 구간(모든 종류)	위와 같음	위와 같음	위와 같음	1 9 + round cap

- 흐름선은 지도 좌표에 고정되어 줌·이동에 맞춰 움직입니다. 화면 중앙 고정 장식 화살표는 쓰지 않습니다.
- 애니메이션은 넣지 않았습니다(정지 상태에서 방향이 명확함).
- 필터: `odor` 선은 `filter != 'all'` 이고 `pred.type != filter` 이면 숨깁니다. `wind` (중립) 선은 필터와 무관하게 표시합니다.

13.9 나침반 카드 #compass

구성(위→아래): 1. 머리줄 버튼(접기/펼치기, `aria-expanded`): 제목 `#compass-title` = gps면 내 위치, 아니면 구역 공개 이름(구역 밖이면 선택 위치) + 상태 배지 `#compass-status`. 2. 나침반 SVG(160 viewBox, 데스크톱 140px): 바깥 원 반지름 64(#f8fafc, 선 #d7e1e8), 안쪽 점선 원 48, 북(위)·동·남·서 11px 회색 글자. 바늘 `#needle`: 앞쪽 M80 24L89 85L80 79L71 85Z (currentColor), 뒤쪽 회색 #cbd5e1, 중심 흰 원 반지름 5에 파란 테두리. 회전은 CSS `transform: rotate(<누적각>deg)`, `transform-box:view-box`; `transform-origin:80px 80px`; `transition: transform .45s ease`(최단 호 누적, §8.6). 3. 들어오는 방향(12px 회색 라벨) 4. `#direction`(16px 굵게) 5. `#multi` 배지 여러 방향 가능(해당할 때만) 6. `#type-label` 7. `#model-badge`(예비 실험·미보정 또는 추정 보류) 8. `#compass-at` = HH:MM 기준(오늘) 또는 MM.DD HH:MM 기준(오늘이 아님) 9. `#timing-label` = 선택 시각 기준, 예보 구간이면 예보 기준

상태별 표시 (선택 구역의 `arrow` 로 결정. 구역 밖이면 바람 유무로 `wind / calm / none`):

arrow	상태 배지	바늘	direction	type-label
odor	유입 가능성 높음/보통/낮음 (high/medium/low 색)	보임, 유형색, <code>wind.from</code>	동북동쪽에서 유입 추정 (pred.from_sector)	축산계 · 추정
wind	level이 있으면 유입 가능성 ..., 모델 없으면 추정 보류	보임, 회색	동북동쪽 바람	바람
calm	level 배지	숨김(.off), 나침반 흐리게(.55)	방향 보류	악풍
none	자료 없음	숨김	방향 미확인	프레임이 있으면 자료 부족, 없으면 자료 없음

- 나침반 SVG의 `aria-label`: 북쪽 고정 나침반. 동북동쪽에서 들어옴 또는 북쪽 고정 나침반. 방향 표시 없음.
- 나침반과 지도는 **같은 위치·같은 시각·같은 계산 결과**를 씁니다.

13.10 시간 바 `#timeline`

- 머리줄: `#time-label` MM.DD HH:MM KST (14px 굵게) · `#time-mode` 배지 · `#basis` 배지 · `#future` (11px) · 오른쪽 끝 최신 버튼(파란 굵은 글씨).
- 조작줄: <(이전) >(재생, 재생 중 II / `aria-label` 정지) 슬라이더(`input type=range`, step 1, 0~프레임수-1, `accent-color:#1F5C8B`, 높이 44) >(다음).
- `#time-mode` 값:
- 프레임 없음 → 자료 없음
- 예보 프레임 → `data.forecast.label` (시연 예보 / 예보), forecast 배지 스타일
- `latest_index` 프레임이고 `now - at > 1시간` → 업데이트 지연(hold 스타일)
- `latest_index` 프레임이고 1시간 이내 → 최신
- 그 외 → 지난 기록
- `#basis`: 관측 프레임은 관측 / 추정 / 시연, 예보 프레임은 예보 기준.
- 슬라이더 `aria-valuetext` = "MM.DD HH:MM <모드>" .
- 이동 규칙:
- 슬라이더·이전·다음은 재생을 멈추고 해당 프레임으로 이동합니다.
- 모드는 자동 결정: 예보 프레임 → forecast, `latest_index` → latest, 그 외 past.
- 최신은 알림을 닫고 `latest_index` 로 이동합니다(예보 끝이 아님).
- 재생: 마지막 프레임이면 처음으로 가서 시작하고, 1.8초마다 다음 프레임, 끝에서 멈춥니다.
- 자동 갱신(15초 캐시)이 과거를 보던 사용자를 최신으로 옮기지 않습니다. `state.at` 을 유지합니다.
- 이전/다음은 양 끝에서 비활성, 재생은 프레임이 2개 미만이면 비활성.

13.11 위치 선택

- ✦ 위치 선택: 선택 모드를 토글(버튼 active)하고 알림 지도에서 위치를 선택하세요 를 띄웁니다. 이 모드에서만 지도 클릭이 기준 위치를 바꿉니다.
- 선택 지점이 김포 경계 밖이면 위치를 바꾸지 않고 알림 김포 밖입니다 · 운양동에서 위치 선택.
- ☉ 내 위치:
- geolocation이 없으면 위치 사용 불가 · 지도에서 선택.
- 있으면 알림 위치 확인 중 후 `getCurrentPosition(timeout 10s, maximumAge 60s, 고정밀 아님)`.
- 성공하고 김포 안이면 `position_kind='gps'` 로 선택하고 `setView(pos, 14)`. 김포 밖이면 위 김포 밖 알림.
- 거부·실패하면 위치 권한 또는 연결 확인 · 지도에서 선택.

- 요청마다 번호를 매겨 **늦게 온 응답은 무시합니다**(그 사이 다른 선택을 했으면).
- 초기 기본 지점을 "내 위치"라고 부르지 않습니다(항상 기준 위치/선택 위치).
- 선택
위치: 기준 위치로 `setView(position, 14)`. 김포
전체: 김포 경계에 `fitBounds(padding 20)`.
- `Esc`: 재생 정지, 선택 모드 해제, 알림 숨김, 패널 닫기.

13.12 요약 #summary (위치 카드 4줄)

- 관측 프레임: [관측 상태 배지] + (n>0이면) 감지 3/7명 (aria-label="7명 중 3명 감지").
- 예보 프레임: [예보] (forecast 배지) + 유입 가능성 보통.
- 관측 상태(주민)와 모델 추정(나침반)의 역할을 분리합니다. 범례의 마커 관측 · 화살표 추정 이 이를 설명합니다.

13.13 범례 레이어 라벨 #layer-label

- 필터에 걸려 숨겨졌으면 선택 유형 없음.
- odor → 축산계 · 이동 방향, wind → 바람 이동, calm → 방향 보류, none → 방향 미확인.
- 예보 구간이면 끝에 · 점선 예보를 붙입니다.

13.14 상세 패널 (내용 전체)

공통: `rows()` = 2열 정의 목록(dt 90px 12px 회색 / dd 13px). 모델 결과가 있는 패널에는 예비 실험·미보정 배지를 넣습니다. 도움말은 <details>(클릭형, 모바일에서도 동작. hover 방식 사용 안 함).

1. **공기질 관리** (air) - 행: 등록 위치(구역 이름, 구역 밖이면 선택 위치 · 관측 구역 밖) · 주변 냄새(outlook 문장, 구역 밖이면 자료 없음) · 자동 대응(사용 안 함이면 사용 안 함, 아니면 켜기 조건 충족 · 높음 2회 연속 / 해제 조건 충족 · 낮음 2회 연속 / 유지 · 조건 미충족) · 기기 연결된 기기 없음 - 그다음: 예비 실험·미보정 배지, 체크박스 자동 대응 사용 안 함(state.auto_enabled 반영 → Python이 다시 판단), 비활성 버튼 기기 연결 · 준비 중 - 자세히 안: 공기질 관리미 · 판단 결과만 표시합니다. 기기 제어는 없습니다. / 켜기: 유입 가능성 높음 2회 연속 · 해제: 낮음 2회 연속 · 그 외 유지 - 전원 토글이나 연결 성공 표시는 없습니다.
2. **냄새 기록** (record): MEASUREMENT_APP_URL 이 http(s)이면 지금 느낀 냄새를 기록하세요. + 파란 링크 버튼 냄새 기록 열기 ↗ (새 탭, noopener). 아니면 기록 연결 · 준비 중.
3. **구역 상세** (zone, 제목=구역 이름): 위치(내 위치/선택 위치) · 집계 구역(가까운 관측 구역 · 경계 미확정 / 관측 구역 밖) · 관측(상태, 예보면 관측 없음 · 예보) · 감지(7명 중 3명 / -) · 중앙 강도 · 추정(유입 가능성 ... / 추정 보류 / -) · 방향(방향 보류/direction) · 기준 시각 · 구역 대표점 임시 위치 + 배지 + 버튼 집계 기준 ①.
4. **흐름선 상세** (flow, 제목 유입 추정 / 유입 추정 · 예보 / 바람 이동): 유형(type_label 또는 유형 미확인) · 들어오는 방향 동북동쪽 · 67.5° · 이동 방향 서남서쪽 · 247.5°(약풍이면 방향 보류) · 방향 판단 · 정렬 후보(odor일 때만 이름(유형), ..., 점수 없음) · 기준 시각 · 자료 구분(관측 / 시연 / 예보 · 시연 예보) · 풍속 0.5 m/s + 배지 + <details>추정 방향 ①: 정렬 후보는 방위 일치이며 발생원 판정이 아닙니다. / 화살표 한 방향을 여러 번 그린 참고선입니다. 확산 경로가 아닙니다.
5. **자료 정보** (info): payload.info 를 표로 + 경계 출처 ↗ 링크.
6. **집계 기준** (criteria): - 정기·실외 관측 중 냄새 유무를 판단한 응답만 집계합니다. 판단 어려움, 실내, 추가 제보, 대응 후 평가는 분모에서 제외합니다. - 같은 참여자의 같은 구역·시간 기록은 마지막 응답을 사용합니다. 유효 응답 3명 미만은 자료 부족입니다. - 화살표는 추정이 낮고 실제 감지율도 25% 미만이면 바람 이동(회색)만 표시합니다.
7. **도움말** (help, 기본): 버튼 자료 정보 →, 집계 기준 →, 그리고 세 개의 <details>: - 추정 방향: 지도는 이동 방향, 나침반은 들어오는 방향입니다. 북동풍이면 지도는 남서쪽, 나침반은 북동쪽을 가리킵니다. / 마커 색: 주민 관측 · 화살표: 모델 추정(예비 실험·미보정). 화살표 색은 추정 유형이며 위험 등급이 아닙니다. /

주황 실선: 축산계 · 보라 긴 점선: 하수계 · 회색: 미확인 또는 바람 이동 · 둥근 점선: 예보. - 위치 ·
개인정보: 내 위치는 버튼을 눌러 권한을 허용할 때만 확인합니다. 선택 좌표는 조회 계산에만 쓰고 관측
DB에 저장하지 않습니다. / 2km 이내의 가장 가까운 관측 구역을 참고합니다. 건물별 예측이 아닙니다. -
시간 · 자료 상태: 지난 기록은 직접 고른 과거 시각입니다. 최신 자료가 1시간 넘게 오래되면 업데이트
지연입니다. / 최신 이후는 예보 구간입니다. 기상청 연결이 없으면 시연 예보입니다. / 자료 없음·자료
부족·방향 보류는 냄새 없음이 아닙니다. - 메뉴 active 표시: 현재 패널이 air/record/help면 그 버튼, 그 외
(지도 계열 zone/flow/info/criteria/없음)는 지도 버튼. - 지도 메뉴는 패널을 닫습니다. 패널을 열어도
지도의 위치·시간 선택은 유지됩니다.

14. 문구·용어 사전 (네이버지도식 짧은 표현)

원칙: 지도 메인에는 **메뉴명·상태·방향·시각·필요한 조작만** 둡니다. 설명 문단을 지도 위아래에 늘어놓지 않습니다. 메뉴는 짧은 명사, 버튼은 짧은 동작입니다. 같은 안내를 여러 카드에서 반복하지 않고, 긴 설명은 **도움말·자료 정보·집계 기준** 으로 보냅니다.

쓰지 않는 표현	쓰는 표현
냄새 지도	지도
공기질 관리미 (메뉴)	공기질 관리 (서비스명 "공기질 관리미"는 패널 '자세히' 안에서만)
냄새 기록 참여	냄새 기록
이용 안내	도움말
내 위치 사용	내 위치 (실제 권한으로 확인한 지점에만 사용)
지도에서 기준 위치 선택	위치 선택
선택 위치로 복귀	선택 위치
김포 전체 보기	김포 전체
최신 자료로 돌아가기	최신
자료 출처 및 기준 안내	자료 정보
관측 자료가 부족합니다	자료 부족
이 시각의 자료가 없습니다	자료 없음
최신 기상자료가 지연되었습니다	업데이트 지연
과거 시점의 자료입니다	지난 기록
시나리오 기반 가상 자료입니다	시연
냄새 유형을 확인하기 어렵습니다	유형 미확인 / 미확인
장치가 연결되어 있지 않습니다	연결된 기기 없음
향후 제공할 예정입니다	준비 중
기상자료 지연 · 과거 기준 시각의 바람입니다	업데이트 지연 + 09.16 22:00 기준
풍향과 주민 관측의 관계를 보여주며 발생원을 판정하지 않습니다	범례 추정 방향 ① (설명은 상세로)
개인 위치는 표시하지 않습니다	메인에서 제거 → 도움말 "위치 · 개인정보"
시연 자료: ...655건...simulated...	시연 배지 → 자료 정보
구역 대표점은 임시 설정입니다...	구역 상세 구역 대표점 · 임시 위치 / 운영자 좌표 화면
정기 관측 07:30...각 ±15분	메인에서 제거 → 기록 앱 참여 안내
판단 어려움·실내 기록·추가 제보는...제외	집계 기준 ①

- 관측 결과는 감지 3/8명(접근성 라벨 8명 중 3명 감지), 예측 결과는 유입 가능성 높음. **예측만 있는 상태를 "냄새 감지"라고 쓰지 않습니다.**
- 방향 문구: 북동쪽에서 유입 추정 / 바람만 있으면 북동쪽 바람 / 방향 보류 / 방향 미확인 / 여러 방향 가능.

- 시각 문구: 오늘이면 14:30 기준, 아니면 09.16 22:00 기준.
 - 필터 그룹 라벨 냄새 유형 . 추정. 버튼마다 "추정"을 반복하지 않습니다.
 - 개발 명령, 내부 변수명, 운영자 검증 지침은 주민 화면에 보이지 않아야 합니다.
-

15. 주민 기록 앱 "냄새 기록" (measurement_app.py)

- `st.set_page_config(page_title='냄새 기록', layout='centered')`. 공용 CSS 헤더에 냄새 기록 제목(시연 배지 없음).
- 상단 가로 라디오 메뉴: 새 관측 · 내 최근 기록 · 참여 안내.
- 저장소 연결 안내 캡션: 공유 저장소에 기록됩니다. / 이 컴퓨터에 기록됩니다. / 현재는 조회 시연만 가능합니다. 기록 저장 연결을 준비 중입니다. / 예외 시 서버 연결 실패 - 저장소 설정을 확인 중입니다.
- **참여 안내**: 정기 관측은 07:30 · 12:30 · 18:30 · 22:00 전후 15분입니다. 다른 시간에는 추가 제보로 자동 기록됩니다. / 냄새가 없는 기록도 중요합니다. 기억으로 채우지 말고 지금 확인한 상태를 선택해 주세요. / 캡션 서버 연결이 필요하며 오프라인 저장·재전송은 제공하지 않습니다. 코드와 구역은 이 세션에서만 기억합니다.
- **처음 한 번**(세션에 참여자가 없을 때): 소제목 처음 한 번만 알려주세요, 폼에 참여자 코드 (placeholder 예: R07, 정규식 `[A-Za-z0-9_-]{2,24}`, 대문자로 저장) + 관측 구역 선택(기본값 없음, 공개 이름으로 표시) + 버튼 관측 시작. 잘못되면 참여자 코드와 관측 구역을 입력하세요.
- 그 뒤 캡션 <구역 이름>에서 관측합니다 · <코드>.
- **새 관측 단계**(진행 막대 step/4, 캡션에 정기/추가 제보 안내). 모든 라디오에 기본값이 없고, 선택 전에는 다음 이 비활성입니다. 1. 1 · 지금 어디에서 확인하고 있나요? — 관측 환경: 실외 / 실내 → 다음 2. 2 · 지금 냄새가 느껴지나요? — 냄새 확인: 느껴짐 / 느껴지지 않음 / 판단하기 어려움 → 다음 (느껴짐이면 3단계, 아니면 4단계) 3. 3 · 냄새의 느낌을 알려주세요 — 강도 1~5(1 · 거의 느끼기 어려움, 2 · 약함, 3 · 분명함, 4 · 강함, 5 · 매우 강함), 냄새 느낌: 분노와 비슷함 / 하수구와 비슷함 / 탄내 / 화학물질과 비슷함 / 기타 / 구분하기 어려움 → 내용 확인(둘 다 골라야 활성화) 4. 마지막 · 내용 확인 — 관측 구역, 관측 환경, 냄새, (느껴짐이면 강도·느낌), 캡션 관측 시각: MM/DD HH:MM KST → 기록 저장(primary, 저장소가 없으면 비활성)
- 2단계부터 이전 단계 버튼(4단계에서 느껴짐이 아니었으면 2단계로). 돌아가면 초안을 폐기합니다.
- **저장**:
- 초안은 4단계에 처음 들어올 때 `build_observation`으로 만들고, 맥등 키는 세션의 `submission_key` (UUID4)입니다.
- 첫 시도 때 `received_at`과 `collection_mode`를 그 순간 기준으로 확정합니다.
- 실패하면 서버 연결 실패 - 저장을 확인하지 못했습니다. 같은 기록으로 다시 시도해 주세요. 를 띄우고 초안을 유지합니다. 재시도해도 같은 키라 중복이 생기지 않습니다.
- 성공하면 세션의 `own_ids`에 추가하고 입력값을 초기화한 뒤 기록되었습니다., 냄새가 느껴지지 않은 기록도 중요한 자료입니다., 버튼 새 관측 시작.
- **내 최근 기록**: 저장소에서 참여자 코드로 읽되 이 세션에서 제출한 ID만 표시(MM/DD HH:MM · 느껴짐/느껴지지 않음/판단 어려움). 없으면 이 세션에서 제출한 기록이 없습니다. 캡션 참여자 코드는 인증 수단이 아니므로 다른 세션의 개별 기록은 공개하지 않습니다.
- **표준 레코드** `build_observation(participant, zone, context, odor, intensity, odor_type, key, moment=None)`:

```
observation_id = key (UUID)                participant_id, zone_id(Z1~Z3)
observed_at = KST ISO                      received_at = now KST ISO
context = outdoor|indoor                  odor_detected = True|False|None
intensity = 1~5(True) / 0(False) / None(None)
odor_type = 선택값(True) / '없음'(False) / '판단 어려움'(None)
collection_mode = scheduled|spontaneous (§8.1) prediction_seen = False
window_id = observation_window(moment).isoformat() record_origin = 'resident' idempotency_key = key
```

- 검증: odor가 True/False/None이 아니거나 context가 잘못되면 관측 항목을 선택하세요. , True인데 강도·유형이 없으면 강도와 냄새 느낌을 선택하세요.
 - 조회 응용 변환 `legacy_reports(rows)`: 열 이름을 `report_id`, `observer_code`, `submitted_at`, `environment`, `odor(1/0/NaN)`, `report_mode`, `saw_forecast`로 바꾸고, 시각은 KST naive로 맞춥니다.
-

16. 관측 저장소

`observation_repository()` 선택 규칙: 1. `database.url` 또는 `database.key` 가 있으면

SharedObservations(Supabase REST). 문구 공유 저장소에 기록됩니다. 2. 없고 `APP_ENV == 'local'` 이면

LocalObservations(`OBSERVATION_DB_PATH`). 문구 이 컴퓨터에 기록됩니다. 3. 그 외(클라우드인데 공유 저장소 없음) → `None`. 저장 버튼 비활성, 저장된 척하지 않습니다.

LocalObservations (SQLite): 테이블 `observations(idempotency_key TEXT PRIMARY KEY, participant_id, observed_at, payload JSON)`. `INSERT OR IGNORE` (먹등). 조회는 `observed_at` 내림차순.

SharedObservations (Supabase PostgREST, 서버에서만 호출): - URL `https://.../rest/v1/observations`, 헤더 `apikey` 와 `Authorization: Bearer <service role>`, timeout 15초. `https://` 가 아니면 설정 오류로 처리합니다. - 저장: `POST ?on_conflict=idempotency_key, Prefer: resolution=ignore-duplicates,return=representation`. - 조회: 500건씩 페이지로 읽고, 참여자 필터는 `participant_id=eq.<코드>`. - 네트워크 오류는 서버 연결 실패 - 잠시 후 다시 시도해 주세요.

`deployment/supabase.sql` (운영자가 직접 실행):

```
create table if not exists public.observations (
  observation_id uuid primary key,
  participant_id text not null,
  zone_id text not null check (zone_id in ('Z1','Z2','Z3')),
  observed_at timestamptz not null,
  received_at timestamptz not null,
  context text not null check (context in ('indoor','outdoor')),
  odor_detected boolean,
  intensity integer check (intensity between 0 and 5),
  odor_type text not null,
  collection_mode text not null check (collection_mode in ('scheduled','spontaneous','followup')),
  prediction_seen boolean not null default false,
  window_id text not null,
  record_origin text not null check (record_origin='resident'),
  idempotency_key uuid not null unique
);
alter table public.observations enable row level security;
revoke all on public.observations from anon, authenticated;
grant all on public.observations to service_role;
create index if not exists observations_participant on public.observations(participant_id, observed_at desc);
```

시나리오 CSV는 이 API로 절대 쓰지 않습니다. 주민 제출과 시나리오를 섞거나 덮어쓰지 않습니다.

`core/public_data.py`: - `dashboard(mode)`: `zones.csv`, `sources.csv`를 읽습니다. demo면

`data/provided/reports.csv` 와 정규화한 `weather.csv`, live면 저장소 기록(`legacy_reports`)과 빈 기상. 반환은 (`reports`, `weather`, `zones`, `sources`, `windows=make_windows(...)`, `source_geometry`). - `window_slots(mode)`: demo이고 `windows.csv` 가 있으면 364개 시각을 시간순으로. 아니면 빈 표.

17. 운영자 앱 (admin_app.py)

17.1 인증 (core/auth.py)

- 해시 형식 `pbkdf2_sha256$600000$<salt hex 16B>$<sha256 hex>`. 생성은 `python -m core.auth` (숨김 입력).
- 검증은 `hmac.compare_digest` 로 하고, 반복 횟수는 10만~200만만 허용합니다.
- 해시 설정이 없으면 관리자 인증이 설정되지 않아 운영자 기능이 잠겨 있습니다. 경고 후 중단합니다(데이터 접근 전).
- 로그인 화면: 제목 운영자 로그인, 비밀번호 (password), 로그인. 실패하면 인증 정보를 확인하세요. 성공하면 세션에 저장된 해시의 sha256을 표식으로 저장합니다.

17.2 공통

- 운영자 DB `ADMIN_DB_PATH` (기본 `data/odor.db`, SQLite WAL). 테이블은 `reports`, `weather`, `zones`, `sources`, `model_runs`, `device_log` 이고 `is_sample` 로 샘플과 실데이터를 분리합니다.
- 첫 실행 때 샘플 영역이 비어 있으면 `data/provided` CSV(없으면 `sample_data` 생성기)로 채웁니다.
- 상단: 로고 워드마크 + 운영동 · KST · 운영자 전용. 샘플 모드면 주황 배너 제공 시나리오(가상) 데이터 - 실제 주민 관측 결과가 아닙니다.
- 페이지(가로 라디오): ④ 분석·방향 · ⑤ AI 실험 · ⑥ 우리 집 대응 · ⑦ 예측 모델 · ⑧ 설정.
- 하단 캡션: 냄새 나침반 · 유입 방향 추정은 원인 시설 판정이 아닙니다 · 관측 부족은 감지 없음이 아닙니다.

17.3 ④ 분석·방향 (가설 1·2)

- 카드: 관측 기간, 전체 기록(유효 관측창 수), 집단 감지 창(미정 수). 구역별 응답 수·참여자·응답률.
- H1: 시각별 감지율 막대(표본 수 선 겹침), 감지/미감지별 기상 상자그림(ws, humidity, pressure, temp).
- H2: 8방위 감지율 극좌표 막대. 후보별 정렬도 요약($\text{align} > 0.7$ 일 때 감지율 vs 기타, 배수, 표본). 8방위 카이제곱 p값.
- 자동 결론: 감지율이 가장 높은 방위는 $X(n=)$ 입니다. 후보 D1 정렬도가 높을 때 감지율은 N배입니다. 이는 유입 가능성을 시사하지만 원인 시설을 확정하지 않습니다.
- 집계 CSV 내려받기.
- (이 페이지의 정렬도는 기존 방식 $\cos(\text{wd} - \text{bearing})$ 이고 풍속 0.5 미만은 NaN입니다.)

17.4 ⑤ AI 실험 (기존 모델, core/model.py)

- 학습 조건: 양성 창 10개 이상, 두 클래스 존재, 날짜 2일 이상.
- 마지막 날짜로 평가합니다.
- 비교 대상:
- 기준선 1: 다수 클래스
- 기준선 2: 방향 규칙 `align_D1 > 0.7 & ws < 3`
- 로지스틱 회귀($C=1$, balanced)
- 랜덤포레스트(200개, depth 4, balanced)
- 공통 전처리: 수치는 중앙값 대체 + 표준화, 구역은 원-핫
- 버튼 모델 학습 / 새 기록으로 재학습 → `data/models/admin.joblib` 에 저장하고 `model_runs` 에 기록합니다.
- 성능표, 혼동행렬 히트맵, 변수 영향도를 보여줍니다.
- 캡션: 정렬도 순위는 연관 신호일 뿐 독립 증거가 아님.

17.5 ⑥ 우리 집 대응 (기기 시연)

- 가상 기기 / 아두이노 (시리얼 포트가 있을 때만). 포트가 없으면 배지 실기기 미연결 - 시뮬레이션.
- 설정: 야간 소음 제한(22~07시), 최대 지속시간(5~120분), 수동 중지, 시작 임계값(0.50~0.95, 기본 0.65), 해제 임계값(기본 0.45).
- `device_command` 우선순위: 수동 중지 → 야간 → 최대 시간 → (켜짐이고 $p \leq$ 해제 → OFF) → (꺼짐이고 $p \geq$ 시작 → ON) → 유지.
- 현재 위험도로 명령 평가 버튼을 누를 때만 평가합니다(백그라운드 자동 제어 없음). 응답을 확인하지 못하면 꺼짐으로 간주하고 경고합니다. `device_log` 에 기록합니다.
- 대응 후 평가(10분 후 실내 냄새가 줄었나요?)는 학습 데이터와 분리해 저장합니다.
- Arduino 스케치 `ext/arduino/odor_fan.ino` (9600bps):
- 명령 `RISK:LOW|MEDIUM|HIGH` (LED 초·노·빨, HIGH면 팬 ON), `FAN:ON|OFF`
- 받은 명령마다 `ACK:<명령>` 으로 응답하고, 2초마다 `GAS:<A0 값>` 을 보냅니다
- 핀: FAN 9, G 5, Y 6, R 7, GAS A0
- 이 페이지는 운영자 시연용입니다. 주민 앱에는 기기 제어가 없습니다.

17.6 ⑦ 예측 모델 (odor_model 관리)

- 상단 카드 3개:
- 주민 화면 모델: 번들 또는 run 이름, 예비 실험·미보정
- 로딩 상태: 정상 또는 추정 보류 · 사유, 경로
- 높음 임계값: 값, 운영 설정 · 공인 기준 아님
- 탭 평가:
- 배지 번들 평가 · `odor_model/artifacts` + `evaluation.md` 를 마크다운 그대로 렌더링합니다.
- 재학습 run마다 접힘 영역 재학습 평가 · `<run>` (적용 중이면 · 주민 화면 적용 중)을 둡니다. 기존 평가와 구분합니다.
- 탭 재학습:
- 학습 자료 선택(data/provided/*.xlsx), 캡션 새 모델은 `data/models/odor/<시각>`에 따로 저장됩니다. 아래에서 선택해 적용하기 전까지 주민 화면은 바뀌지 않습니다.
- 모델 재학습 → 하위 프로세스 `python -m odor_model.train --data <xlsx> --out data/models/odor/YYYYmdd-HHMMSS --geo data/geo/model_geo.json (timeout 900초, 약 8초 걸림)`. 성공하면 재학습 완료 · `<run>` · 기존 평가와 별도로 저장했습니다. , 실패하면 `stderr` 끝부분.
- 주민 화면에 쓸 모델 선택(번들 모델(`odor_model/artifacts`) / 재학습 · `<run>`) + 주민 화면에 적용 (현재와 같으면 비활성). 적용 전에 `load_model(choice)` 로 시험 로딩하고, 실패하면 적용하지 않았습니다 · `<사유>`. 성공하면 설정의 `active_model` 을 저장하고 적용했습니다. 조회 앱은 다음 새로고침부터 이 모델을 씁니다.
- 탭 운영 설정:
- 배지 운영 설정 · 공인 기준 아님.
- 입력: 보통 임계값(0.05~0.95), 높음 임계값(0.05~0.99, 기본 모델 `thr_valid`), 약풍 기준 m/s(0~3), 정렬도 최소(0~1), 바람 레이어 전환 감지율(0~1), 자동 대응 켜기 높음 연속 횟수(1~6), 해제 낮음 연속 횟수(1~6).
- 값을 바꾸면 경고 운영 설정·공인 기준 아님 · 저장하면 주민 화면의 표시 기준이 바뀝니다. 보통 $\geq$ 높음이면 오류를 띄우고 저장을 막습니다.
- 설정 저장.
- 탭 좌표:
- 배지 임시 좌표, 캡션 구역 대표점과 후보 지역은 운영자가 검증해야 합니다. 후보는 지역·시설 유형 수준으로만 적습니다. 후보 ID는 모델 특징과 연결되어 바꿀 수 없습니다.

- 구역·후보 표 편집(id, app_zone_id, app_source_id는 읽기 전용) → 좌표 저장 (provisional 유지, zones.csv 동기화).

17.7 ⚙ 설정

- 데이터 소스 샘플 / 실데이터 전환.
 - 샘플이면 캡션 제공 워크북 2026-06-18~2026-09-16 시나리오 · 참여자 20명 · 원본 record_origin=simulated 와 샘플 데이터 재생성 (data/provided CSV로 DB 재구성) 버튼.
 - 실데이터 csv 가져오기: 대상 테이블 선택, UTF-8 또는 CP949 자동 판별, 기상은 기상청 열 이름 별칭 정규화(지점, 일시, 풍향(deg), 풍속(m/s), 기온(°C), 습도(%), 현지기압(hPa), 강수량(mm)).
 - 경고: 공개 클라우드의 로컬 SQLite는 임시 저장소라는 점. 개인정보 원칙 안내.
-

18. 보안·개인정보

- Supabase 키는 서버(Python)에서만 쓰고 화면·URL로 보내지 않습니다. 테이블은 RLS를 켜고 anon/authenticated 권한을 회수합니다.
 - 비밀값 파일(.streamlit/secrets.toml)은 Git에서 제외합니다.
 - 컴포넌트 HTML에 넣는 모든 문자열은 `esc()` 로 이스케이프합니다(& < > " ').
 - 외부 링크는 `target=_blank rel=noopener` . 기록 앱 URL은 `http(s)` 로 시작할 때만 링크로 만듭니다.
 - 공개 앱에는 운영자 앱 링크가 없습니다.
 - 브라우저 페이로드에는 집계·예측 등급만 들어갑니다(\$10.7 확인 목록).
 - 참여자 코드는 인증이 아닙니다. 개인 기록은 현재 세션 제출분만 보여줍니다.
 - 기록 앱에 실명·주소·연락처 입력란이 없습니다.
-

19. 테스트와 검증 기준

19.1 자동 테스트 (python -m pytest -q, 총 37개)

- tests/test_core.py, test_observations.py, test_shared.py: 관측 레코드, 먹등 저장, Supabase 요청 형식, 정기 관측 판정 등(기준).
- tests/test_map.py:
- 프레임 시각 중복 없음, 개인 코드 미포함, wind.to == (from+180)%360, 기상은 프레임 시각 이전 1시간 이내
- 기본 선택 구역 Z2, 김포 밖 좌표는 기본 위치로 복귀, (37.7,126.6)은 구역 밖
- 기상 없으면 wind=None, odor가 None 3건이면 자료 부족, 기록 없는 구역은 자료 없음
- 하수 유형 판정은 감지 2명 이상일 때만
- tests/test_app.py (Streamlit AppTest, 스크립트 경로는 테스트 파일 기준 절대경로):
- 조회 앱 예외 없음·라디오 없음·컴포넌트 1개
- 운영자 잠금·로그인 후 5개 페이지 예외 없음
- 기록 앱의 기본값 없음, 냄새 없음 빠른 경로, 판단 어려움+실내, 느껴짐일 때 강도·유형 필수, 저장 실패 시 초안 유지 후 재시도 성공
- live 조회가 로컬 공유 DB를 읽음
- tests/test_model_service.py (모델을 불러올 수 없는 환경이면 skip): 1. 슬라이더 = 364 관측 프레임(06-18 07:30 ~ 09-16 22:00) + 이후 시연 예보 프레임, latest_index 363 2. 기상 구분이 {관측, 추정, 시연}으로 보존됨 3. 09-11 22:00: 풍속 0.2, Z2 집단 감지, 방향 보류(약풍), 모든 구역 arrow calm 4. 09-15 22:00: 67.5→247.5, Z2 odor·livestock·동북동, 후보 이름 포함 5. 09-14 12:30: window_rate 0, 모든 구역 level 낮음·arrow wind 6. 페이로드에 p_detect, type_probs, align_score, observer_code, % 없음, 후보에 점수 없음 7. 모델 파일 없음 → 전 행 추정 보류·모델 없음, arrow는 wind/calm, model.ok False 8. 깨진 joblib → 모델 로딩 실패로 시작하는 사유, 예외 없음 9. 기상청 호출 실패 → 시연 예보 CSV + 기상청 예보 연결 실패 오류 기록, basis forecast_csv 10. 히스테리시스(낮음,높음,높음→ON, 높음,낮음,낮음→OFF, 높음, 높음,낮음→HOLD, 높음,None→HOLD, 사용 안 함→HOLD, 사용 안 함) 11. get_auto_action이 Z1과 U01 모두에 ON/OFF/HOLD 반환 12. outlook이 09.16 22:00으로 시작하고 (예보) 포함 13. 북동 45°, 2 m/s → move_to 225°, 방위 북동
- scikit-learn 1.5.x 환경에서는 모델 테스트가 skip되고 나머지 27개는 통과해야 합니다(추정 보류 경로가 동작한다는 증거).

19.2 브라우저 검증 (Playwright, 조회 앱 8511 실행 중)

python tests/browser_map_check.py — 1440×900과 390×844 각각에서 확인합니다. - 페이지·iframe 가로 스크롤 없음. 시간 바·나침반·위치 이름이 화면 안(세로 스크롤 없이)에 있음 - 관측 프레임 364개, 최초 모드 업데이트 지연(데이터가 9/16까지이므로), 모델 배지 예비 실험·미보정 - 09-11 22:00: 요약에 집단 감지, 방향 보류, 흐름선 머리 0개, 바늘 .off, 툴팁 운영역 인근 · 집단 감지 · 방향 보류 - 09-15 22:00: 동북동쪽에서 유입 추정, 축산계 · 추정, 유입 가능성..., 머리 8~16개, 첫 머리 rotate(247.5deg), 바늘 67.5°, #D97706 선 존재. 흐름선 클릭 → 정렬 후보·예비 실험·미보정, % 없음 - 09-14 12:30: 바람, 선 색은 #64748B 뿐 - 예보(latest+2): 시연 예보, 예보 기준(두 곳), 19 점선(또는 선 없음), 본문에 실시간 없음 - 공기질 관리: 연결된 기기 없음, (예보), 비활성 기기 연결 · 준비 중. 체크하면 사용 안 함 - 이전 → 지난 기록, 지도 중심·줌·위치 변화 없음. 최신 → 업데이트 지연 - 위치 선택 (37.662,126.679) → 한강변·라베니체 인근. 필터 sewage 반영 - 359°→1° 바늘 누적 변화가 정확히 +2° - 김포 밖 클릭 → 김포 밖 알림, 위치 유지. 재생·정지 후 위치 유지 - 위치 권한 거부 모의 → 위치 권한 알림. 김포 전체 → 줌 ≥ 10. 선택 위치 → 중심 복귀 - 타일 요청 차단 → 오류 카드 표시, 다시 시도 → 숨김

- 프레임 기상·구역을 비우면 흐름선 0, 방향 미확인, 자료 없음 - 페이지 JS 오류 0개 - 모델 파일을 치운 상태에서 `python tests/browser_map_check.py` missing: 배지 추정 보류, 09-15 22:00에 추정 보류·바람, 유색 선 없음 - `python tests/browser_check.py`: 위 지도 검사 + 기록 앱(8512, 테스트 DB `artifacts/browser-resident.db`)의 기본값 없음·냄새 없음 저장 흐름

19.3 완료 기준 요약 (수용 테스트)

1. 두 화면 크기에서 지도가 메인이고 기준 위치·시간 바·나침반이 첫 화면에 함께 보인다.
 2. 2~4개 흐름선이 반복 화살표로 보이고 줌·이동과 함께 움직인다.
 3. 위치·시간·필터를 바꾸면 지도·나침반·요약이 일치한다.
 4. 북동풍 45°에서 지도 225°, 나침반 45°. 359→1°에서 반대로 크게 돌지 않는다.
 5. 과거·시연·예보를 실시간으로 표시하지 않는다. 자료 부족과 냄새 없음이 구분된다.
 6. 위치 권한 거부, 김포 밖, 자료 없음, 지도 로딩 실패, 모델 없음, 예보 실패에 대응한다.
 7. 공기질 관리가 연결 상태를 정직하게 보여준다. 기록 앱이 그대로 동작한다.
-

20. 실행·배포

20.1 로컬

```
python -m pip install -r requirements.txt
$env:APP_ENV = "local"; $env:DATA_MODE = "demo"; $env:MEASUREMENT_APP_URL = "http://localhost:8502"
python -m streamlit run viewer_app.py # 조회 (8501)
python -m streamlit run measurement_app.py --server.port 8502 # 기록 (별도 터미널)
python -m core.auth # 운영자 해시 생성 → ADMIN_PASSWO
RD_HASH
python -m streamlit run admin_app.py --server.port 8503 # 운영자
```

- 로컬 세 프로세스는 `data/resident.db` 를 공유합니다.
- 데이터 재생성: `python -m core.import_workbook data/provided/운양동_악취관측_20260618_20260916.xlsx`.
- 모델 재학습(CLI): \$9.8.

20.2 Streamlit Community Cloud

- 같은 GitHub 저장소·main 브랜치로 앱 2개를 만듭니다: 조회 `viewer_app.py` (기존 사이트는 `app.py` 유지 가능), 기록 `measurement_app.py`. Python 3.11.
- 두 앱 모두 `APP_ENV=cloud`, 같은 `[database] url/key` 를 씁니다. Supabase에서 `deployment/supabase.sql` 을 먼저 실행합니다.
- 조회 앱 secrets: `MEASUREMENT_APP_URL` =기록 앱 주소, 실제 주민 기록을 볼 때 `DATA_MODE=live`, 예보에 `[kma]`.
- 운영자 앱은 공개 배포하지 않고 로컬에서 실행합니다.
- 서로 다른 Cloud 앱의 SQLite는 공유되지 않습니다. 두 앱을 운영하려면 Supabase가 필요합니다.
- main에 푸시하면 연결된 앱이 업데이트됩니다.

21. 설계 결정 기록 (왜 이렇게 했는가)

결정	이유
Plotly 지도 → Leaflet 양방향 커스텀 컴포넌트	클릭 위치, 카메라 유지, 브라우저 위치 권한, 좌표에 고정된 흐름선, 시간 재생을 하나의 상태로 관리하고 Python과 주고받기 위해. npm 빌드·지도 API 키가 필요 없음
Leaflet을 저장소에 포함	CDN 장애·오프라인 시연 대비(배경 타일만 인터넷 필요)
모든 화면이 <code>predict_windows</code> 하나를 사용	지도와 나침반이 다른 결과를 보이는 모순 방지
화살표 종류를 Python에서 계산	규칙을 한 곳(<code>arrow_kind</code>)에서 테스트 가능하게
p값 점수를 페이로드에서 제거	보정되지 않은 점수가 확률처럼 읽히는 것을 원천 차단
저장소 좌표 (zones.csv/sources.csv) 우선	지도에 이미 쓰는 좌표와 모델 좌표를 일치. 모델 ID는 특징과 묶여 유지
scikit-learn 1.8.0 고정	번들 pickle 호환. 로딩 시 시험 예측으로 불일치를 조기 감지
행×구역 일괄 예측	364장 추론 32.6초 → 4.1초. 결과는 동일(검증함)
예보 1시간 캐시, 페이로드 15초 캐시	기상청 호출 제한 + 주민 제출 빠른 반영
흐름선 최대 4개·수직 0.3 km 겹침 제거	구역 3곳이 가까워 선이 겹치는 문제. 기준 2~4개
기준 위치 라벨을 오른쪽에	대표점 위 구역 톨팁과 겹침 방지
자동 대응은 판단만	실기기 연결 전 가짜 제어 금지. 인터페이스 <code>get_auto_action</code> 만 준비
재학습 모델은 별도 폴더 + 명시적 적용	검증 전 모델이 주민 화면에 섞이지 않게. 기존 평가와 구분
슬라이더는 워크북 시간창만	없는 시각 생성 금지. 자발 제보 시각이 프레임을 늘리지 않게
window_id 해석 함수 별도	워크북 w-... 와 라이브 ISO 모두 처리. naive는 이미 KST로 간주

22. 알려진 한계와 미완료 항목

- 모든 주민 기록이 가상(simulated)입니다. 6/18~9/8은 기상도 생성값입니다. 평가 수치는 파이프라인 검증용입니다.
 - 기상청 단기에보는 실제 서비스키로 호출해 보지 않았습니다(실패 시 대체 경로만 검증). 격자 nx/ny(55/128)는 운영자가 확인해야 합니다.
 - 구역 대표점·후보 좌표는 임시값이고, 번들 모델은 `DEFAULT_GEO` 로 학습됐습니다. 좌표를 확인한 뒤 재학습하는 것을 권장합니다.
 - 구역 경계 폴리곤이 없어 "2 km 이내 가장 가까운 대표점"을 씁니다. 건물 단위 예측이 아닙니다.
 - 행정경계는 KOSTAT 2018 공개본입니다(최신 법정 경계 아님).
 - 단일 AWS(427 양촌)로는 건물·강 주변 국지풍을 반영하지 못합니다. 원인 시설이나 기여율은 확정할 수 없습니다.
 - 유형 분류기는 약합니다(정확도 0.378 vs 다수 0.366).
 - 실시간 관측 기상 API는 미연결입니다. live 모드는 기상이 없으면 방향·추정을 보류합니다.
 - Supabase 실제 계정 통합 검증, 알림, 오프라인 저장·재전송, 실제 공기청정기 효과 검증, 주민 로그인은 구현하지 않았습니다.
 - OSM 공개 타일은 대규모 서비스 전에 제공 정책·업체를 검토해야 합니다.
 - 운영자 "⑤ AI 실험"의 기존 모델이 scikit-learn 1.5.x로 저장됐다면 1.8.0에서 재학습이 필요할 수 있습니다 (미확인).
 - 운영자 "⑦ 예측 모델"은 AppTest로 예외가 없는 것만 확인했고, 브라우저로 직접 열어 보지는 않았습니다.
-

23. 1차 기획서(2026-09-15) 대비 변경점

항목	1차 기획서	현재
관측 자료	9/15~18, 4일 실측 계획(최대 320건)	3개월 가상 시나리오 7,143건, 364 시간창
정기 관측 시각	07:30-12:30-18:30-21:30	07:30-12:30-18:30- 22:00 (±15분)
분할	15~17일 개발 / 18일 평가	train 6/18~8/25 · valid 8/26~9/8 · test 9/9~9/16
입력 변수	기본: 풍향·풍속·기온·강수·시간·구역 (습도·정렬도는 장기 확장)	습도·정렬도·계절·이동 벡터·무풍 포함 20개
임계값	0.5 고정	0.5 고정 + 검증 자료 F1 최대 0.75(평가 미사용)
단기 예보	장기 확장 과제	기상청 단기예보/시연 예보 구간 구현(점선·"예보 기준")
화면	대시보드형(입력·지도·AI·방향·분석 탭)	지도 중심 3앱 분리, 네이비지도식 짧은 용어, 작은 나침반, 시간 슬라이더
생활 대응	없음(설명문)	"공기질 관리" 패널: 현재·6시간 예보 요약 + 자동 대응 판단 (제어 없음)
기술	Streamlit-SQLite, CSV 우선	+ Leaflet 컴포넌트, Supabase 공유 저장소, 운영자 인증, 재학습·설정 화면
방향 표시	약풍 0.5 m/s 보류	동일 + 여러 방향 가능·정렬 후보 없음·바람 레이어 규칙

부록 A. 짓다 기획서 양식 요약본

칸	내용
프로젝트 이름	냄새 나침반 · 운양동 주민 참여형 악취 진단 AI
사용자와 상황	김포 운양동 주민이 냄새를 느꼈을 때나 창문을 열기 전 휴대폰으로 확인. 참여 주민은 하루 4회 정기 관측(냄새 없음 포함)
문제	냄새가 언제·어느 방향에서 오는지 몰라 대응이 늦고, 기록이 없어 원인 후보를 좁힐 근거가 없음
해결 방법	주민 관측 + 기상 + 후보 좌표를 결합해 지도에 유입 방향·유형 추정과 예보를 표시
제품	지도 중심 모바일 웹앱 (조화·기록·운영자 3앱, Streamlit)
요구사항	① 기본값 없는 기록 앱과 3명 기준 집계("감지 3/8명", 자료 부족/없음 구분) ② 기상만으로 구역별 유입 가능성·유형 추정, 방향 규칙(지도 이동/나침반 유입), 악풍 보류·추정 보류·바람 레이어 규칙 ③ 전체 화면 지도 + 나침반 + 관측·예보 시간 슬라이더, "예비 실험·미보정", % 금지
검증 조건	① 09.15 22:00 화살표 247.5° 주황·나침반 동북동 / 09.11 22:00 집단 감지·방향 보류 ② 시간순 분할로 기준선 3종 대비 평가표·혼동행렬 ③ 모델 없음·예보 실패에도 동작, 1440×900·390×844 한 화면
데이터	익명 코드·구역·시각·냄새 유무/강도/유형, 관측창 집계, 427 양촌 기상(관측/추정/시연), 임시 좌표. 실명·주소·연락처·GPS 미저장
포함	3앱, 로지스틱 회귀 추정, 기상청 예보 연동, 재학습·설정·좌표 편집, 공기질 관리 판단 화면
제외	실기기 제어, 알림, 확률 보정·공식 위험 등급, 발생원 판정, 건물 단위 예측, 주민 로그인

부록 B. 지도 컴포넌트 HTML 골격 (DOM id 목록)

```
<main id="shell">
  <nav id="nav" aria-label="주 메뉴">
    <div class="brand" title="냄새 나침반"><svg viewBox="0 0 40 40"><circle cx="20" cy="20" r="16"/><path
d="M26 10L22 23L10 29L17 17Z"/><path d="M21 24Q28 30 31 23"/></svg></div>
    <button data-panel="map" class="active"><span class="nav-icon">□</span>지도</button>
    <button data-panel="air"><span class="nav-icon">≈</span>공기질 관리</button>
    <button data-panel="record"><span class="nav-icon">+</span>냄새 기록</button>
    <button data-panel="help"><span class="nav-icon">?</span>도움말</button>
    <span class="nav-bottom">냄새<br>나침반</span>
  </nav>
  <section id="workspace" aria-label="김포 냄새 지도">
    <div id="map" aria-label="지도. 위치 선택 버튼을 누른 뒤 지도를 클릭하세요."></div>
    <div id="topbar">
      <div class="card location-card">
        <div class="row"><span class="eyebrow">김포 · 냄새 나침반</span><span id="demo" class="badge">시연</span></div>
        <div class="row location-title"><span class="blue-dot"></span><strong id="location-name">운양역 인근</strong></div>
        <div class="location-actions"><button id="locate">☉ 내 위치</button><button id="pick">✦ 위치 선택</button></div>
        <div id="summary" role="status"></div>
      </div>
      <div class="card filters"><span class="eyebrow">냄새 유형 · 추정</span>
      <div role="group" aria-label="냄새 유형 필터" id="filters">
        <button data-filter="all" class="active">전체</button><button data-filter="livestock">축산계</button>
        <button data-filter="sewage">하수계</button><button data-filter="other">기타</button><button data-filter="unknown">미확인</button>
      </div>
    </div>
    <div id="notice" class="card" role="status" hidden></div>
    <div id="tile-error" class="card" role="alert" hidden><strong>지도 연결 실패</strong><span>인터넷 연결을 확인하세요</span><button id="retry">다시 시도</button></div>
    <div id="map-controls" class="card" aria-label="지도 조작">
      <button id="zoom-in" aria-label="확대">+</button><button id="zoom-out" aria-label="축소">-</button>
      <button id="recenter">선택<br>위치</button><button id="overview">김포<br>전체</button>
    </div>
    <div class="card" id="legend"><button id="legend-help">추정 방향 ①</button><span class="legend-key">마커 관측 · 화살표 추정</span><span id="layer-label">이동 방향</span></div>
    <section class="card" id="compass" aria-label="선택 시각 나침반">
      <button id="compass-toggle" aria-expanded="true"><strong id="compass-title">선택 위치</strong><span id="compass-status" class="badge">자료 부족</span></button>
      <div id="compass-body">
        <svg id="rose" viewBox="0 0 160 160" role="img" aria-label="북쪽 고정 나침반">
          <circle cx="80" cy="80" r="64" fill="#f8f8fc" stroke="#d7e1e8"/><circle cx="80" cy="80" r="48" fill="none" stroke="#d7e1e8" stroke-dasharray="2 6"/>
          <g fill="#657585" font-size="11" text-anchor="middle"><text x="80" y="12">북</text><text x="151" y="84">동</text><text x="80" y="157">남</text><text x="9" y="84">서</text></g>
          <g id="needle"><path d="M80 24L89 85L80 79L71 85Z" fill="currentColor"/><path d="M80 136L73 79L80 85L87 79Z" fill="#cbd5e1"/></g>
          <circle cx="80" cy="80" r="5" fill="white" stroke="#1f5c8b" stroke-width="2"/>
        </svg>
        <div class="compass-copy"><span class="eyebrow">들어오는 방향</span><strong id="direction">방향 미확인</strong>
        <span id="multi" class="badge" hidden>여러 방향 가능</span><span id="type-label"></span>
      </div>
    </section>
  </div>
```

```

    <span><span id="model-badge" class="badge model">예비 실험.미보정</span></span>
    <span id="compass-at" class="muted"></span><span class="muted timing-label" id="timing-label">선택
시각 기준</span></div>
  </div>
</section>
<aside id="panel" class="card" hidden aria-label="상세 정보"><div class="panel-head"><h2 id="panel-ti
tle"></h2><button id="close-panel" aria-label="닫기">×</button></div><div id="panel-content"></div></as
ide>
<section id="timeline" class="card" aria-label="시간 선택">
  <div class="timeline-heading"><strong id="time-label">자료 없음</strong><span id="time-mode" class
="badge"></span><span id="basis" class="badge" hidden></span><span class="muted" id="future">미래 자료
없음</span><button id="latest">최신</button></div>
  <div class="timeline-controls"><button id="previous" aria-label="이전 시각"></button><button id="pl
ay" aria-label="재생">▶</button><input id="time-slider" type="range" min="0" max="0" value="0" step
="1" aria-label="관측 시각"><button id="next" aria-label="다음 시각">></button></div>
</section>
</section>
</main>
<script src="vendor/leaflet.js"></script><script src="map.js"></script>

```

<head>에는 lang="ko", viewport width=device-width,initial-scale=1, vendor/leaflet.css, map.css. 테스트를 위해 window.odorMap = {state, map, data, needleAngle, deltaAngle, direction, flowLines}(읽기 전용 getter) 와 전역 함수 render()를 노출합니다.

부록 C. 재구축 순서 체크리스트

1. 저장소 뼈대, `requirements.txt` (scikit-learn==1.8.0), `.streamlit/config.toml`, `.gitignore` (!odor_model/artifacts/).
2. `core/config.py` (KST·정기 시각·공개 이름·설정 읽기), `core/geo.py`.
3. 원본 워크북 → `core/import_workbook.py` → `data/provided/*.csv` 4개. 364/7,143/364 행 수 확인.
4. `data/sample/zones.csv` · `sources.csv`, `scripts/fetch_map_assets.py` 로 경계·Leaflet 받기.
5. `odor_model/` (features → train → predict). 학습해 `artifacts/` 생성, 평가표가 \$9.6과 비슷한지 확인.
6. `data/geo/model_geo.json`, `data/model_settings.json`.
7. `core/analysis_service.py` → 세 검증 시각의 예측을 콘솔로 확인(\$7.1 표).
8. `core/automation.py`.
9. `core/public_data.py`, `core/features.window_id_time`, `core/map_data.py` (페이로드 ·outlook-resolve_selection) → `tests/test_map.py`, `tests/test_model_service.py`.
10. `components/odor_map/` (부록 B + §13 CSS-JS) + `core/map_component.py` + `viewer_app.py`.
11. `core/observations.py`, `measurement_app.py`, `deployment/supabase.sql`.
12. `core/auth.py`, `core/repo.py`, `core/schema.py`, 기존 분석 모듈, `admin_app.py` (⑦ 포함), `ext/`.
13. pytest 전체 → 8511/8512 실행 → `tests/browser_map_check.py`, missing 모드, `tests/browser_check.py`.
14. 두 화면 크기 스크린샷을 눈으로 확인(겹침, 문구 길이, 배지).
15. README-DEPLOYMENT 작성, 비밀값 예시 확인.